

智能充电桩调度计费系统

系统概要设计

基于 UML 的面向对象建模方法

班 级：2023211301 班

组 长：宋旗舰

组员 1：黄俊恺

组员 2：张艺鸿

组员 3：龙康维

组员 4：谭怡萱

日 期：2026/06/03

目录

1 软件架构	3
1.1 软件架构图	3
1.2 分层结构说明	3
1.2.1 用户界面层	3
1.2.2 控制器层	4
1.2.3 业务/应用层	4
1.2.4 持久化层	5
1.2.5 请求处理流程总结	6
2 系统的界面设计	7
2.1 用户端界面设计	7
2.2 管理员端界面设计	10
3 系统动态结构设计	14
3.1 UC1 注册	14
3.1.1 已知条件	14
3.1.2 对象设计: createNewAccount(car_Id, userName, car_Capacity)	14
3.1.3 对象设计: set_pwd(password)	15
3.2 UC2 充电申请	17
3.2.1 已知条件	17
3.2.2 对象设计: E_chargingRequest(car_Id, Request_Amount, Request_Mode)	17
3.2.3 对象设计: Modify_Amount(car_Id, Amount)	18
3.2.4 对象设计: Modify_Mode(car_Id, Mode)	19
3.2.5 对象设计: Query_Car_State(car_id)	20
3.2.6 对象设计: Start_Charging(car_id, ChargePileNum)	21
3.2.7 对象设计: Query_Charging_State(car_id)	22
3.2.8 对象设计: End_Charging(car_id, ChargingPileNum)	23
3.3 UC3 查看账单	25
3.3.1 已知条件	25
3.3.2 对象设计: Request_Bill(carId, date)	25
3.4 UC4 查看详单	26
3.4.1 已知条件	26
3.4.2 对象设计: Request_DetailedList(Bill_Id)	26
3.5 UC5 运行充电桩	27
3.5.1 已知条件	27

3.5.2	对象设计: powerOn(pile_Id)	27
3.5.3	对象设计: setParameters(peakPrice, normalPrice, valleyPrice, serviceFeeRate)	28
3.5.4	对象设计: Start_ChargingPile(pile_Id)	29
3.5.5	对象设计: powerOff(pile_Id)	30
3.6	UC6 查看充电桩状态	32
3.6.1	已知条件	32
3.6.2	对象设计: Query_PileState(pile_Id)	32
3.7	UC7 查看队列状态	33
3.7.1	已知条件	33
3.7.2	对象设计: Query_QueueState(queueList)	33
3.8	充电桩故障调度策略	34
3.8.1	优先级调度策略	34
3.8.2	时间顺序调度策略 (FCFS)	35
3.8.3	充电中故障恢复策略	36
3.8.4	可选加分: 单次调度总充电时长最短策略	38
3.8.5	可选加分: 批量调度总充电时长最短策略	38
4	系统静态结构设计	40
4.1	UC1 注册	41
4.2	UC2 充电申请	43
4.3	UC3 查看账单	45
4.4	UC4 查看详单	47
4.5	UC5 运行充电桩	48
4.6	UC6 查看充电桩状态	49
4.7	UC7 查看队列状态	51
4.8	系统级静态结构	52
5	工作量统计	58

1 软件架构

1.1 软件架构图

本系统采用 B/S 架构，后端基于 SpringBoot 框架，前端使用 Vue.js 构建用户客户端和管理员客户端。系统划分为用户界面层、控制器层、业务/应用层和持久化层；系统层由 JVM、SpringBoot 运行时、SQLite JDBC 驱动和操作系统文件系统提供支撑。底层采用 SQLite 作为轻量级关系型数据库。图1展示了各层包含的组件及其调用关系。

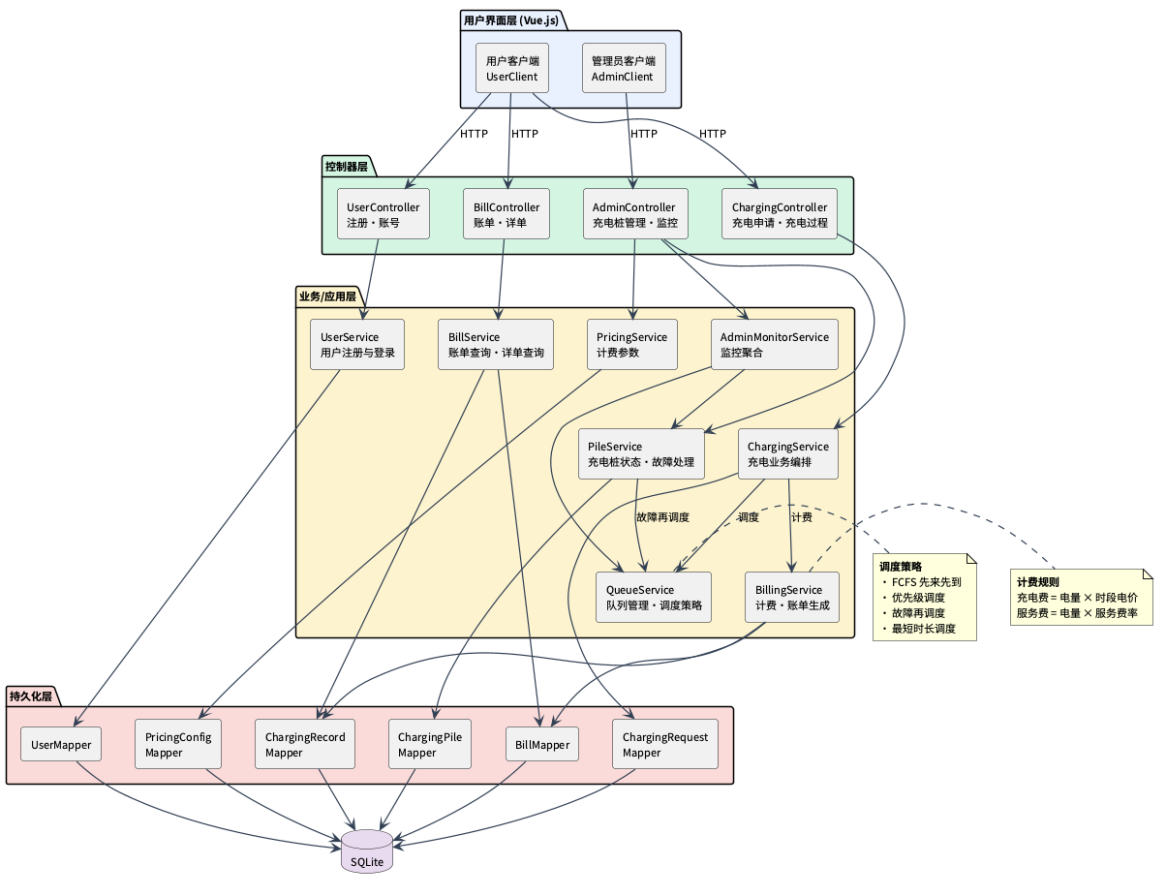


图 1: 智能充电桩调度计费系统软件架构图

1.2 分层结构说明

以下以“用户提交充电申请”为例，沿着请求在系统中的流转路径，逐层说明各对象的职责与协作方式。

1.2.1 用户界面层

用户界面层是系统与用户或管理员直接交互的入口，主要由 Vue.js 前端页面组成。用户端界面负责采集用户注册、充电申请、查询排队状态、查看账单和详单等操

作输入，并展示申请状态、分配桩号、充电进度和费用结果；管理员端界面负责展示充电桩运行状态、等待队列状态和计费参数，并采集启动/关闭充电桩、设置计费参数等管理操作。用户界面层只负责输入采集、结果展示和 HTTP 请求发送，不直接处理业务规则，也不直接访问数据库。

以“提交充电申请”为例，请求的起点是用户在用户端界面（Vue.js 构建的 User-Client）上选择充电模式和期望电量，点击“提交申请”。前端将用户输入封装为 HTTP POST 请求：

```
POST /api/charging/request
{ car_Id: "京A12345", Request_Amount: 40, Request_Mode: 0 }
```

1.2.2 控制器层

控制器层负责接收用户界面层发过来的请求。上述请求到达后端的 **Charging-Controller** 后，控制器层作为系统与用户界面层交互的唯一后端入口，负责接收 HTTP 请求、校验参数合法性（car_Id 是否已注册、Amount 是否在合理范围、Mode 是否有效），然后将请求转发给业务/应用层处理。

本系统中的控制器对象包括：

- **UserController**——接收用户账号相关请求（注册、设置密码）
- **ChargingController**——接收充电申请相关请求（提交申请、修改电量、修改模式、查询车辆状态、开始充电、查询充电状态、结束充电）
- **BillController**——接收账单列表查询和账单详单查询请求
- **AdminController**——接收管理员客户端请求（启动/关闭充电桩、设置计费参数、查看充电桩状态和队列状态）

1.2.3 业务/应用层

业务/应用层负责创建对象实例、实现系统级功能并处理业务规则。**Charging-Controller** 调用 **ChargingService.submitRequest(car_Id, 40, 0)**。业务/应用层是系统功能实现的核心，在这一步完成：

1. **创建业务对象**：调用 `new ChargingRequest()` 创建一个新的充电请求对象，并初始化其属性——car_Id、Request_Amount（40 kWh）、Request_Mode（0=慢充）、request_time（系统当前时间）、car_state（“等待”）。
2. **持久化**：调用 `ChargingRequestMapper.insert(request)` 将该请求保存到数据库。

3. **加入等待队列**: 调用 `QueueService.enqueue(request, “慢充”)`, 将请求按先来先到顺序加入对应充电模式的等待队列, 并计算排队序号。

本系统业务/应用层中的对象可分为三类:

- **Service 对象**: `UserService` 负责用户注册与登录; `ChargingService` 负责充电申请、开始充电、查询充电进度和结束充电等业务编排; `QueueService` 负责等待队列和服务队列管理; `BillingService` 负责费用计算与账单生成; `BillService` 负责账单列表和账单明细查询; `PileService` 负责充电桩状态管理、启动/关闭和故障处理; `PricingService` 负责计费参数查询与设置; `AdminMonitorService` 负责管理员端充电桩状态和队列状态聚合查询。
- **状态机/策略对象**: `ChargingStateMachine` 集中维护充电请求和充电桩状态迁移规则; 调度策略对象用于表达 FCFS、优先级调度、故障再调度等规则。
- **表示业务数据和状态的对象**: `ChargingRequest` (充电请求)、`ChargingRecord` (充电记录)、`Bill` (账单)、`ChargingPile` (充电桩)、`User` (用户)、`PricingConfig` (计费参数) 等对象用于保存业务状态, 并作为 `Service` 对象执行业务规则时的操作对象。

这些对象通过协作完成具体业务要求: 调度策略由 `QueueService` 实现, 当充电区有空余充电桩时, 从等待队列中按 FCFS 顺序取出请求, 根据“使被调度车辆完成充电所需时间最短”规则分配充电桩; 计费规则由 `BillingService` 实现, 按照充电电量、时段电价和服务费率计算费用, 并在充电结束时生成 `Bill` 对象; 故障再调度由 `PileService` 与 `QueueService` 协作, 当充电桩发生故障时, `PileService` 标记故障状态, `QueueService` 对受影响车辆执行再调度。

1.2.4 持久化层

持久化层负责数据的同步和存储。在上述流程中, 所有数据库操作都通过 `Mapper` 对象完成。Mapper 对象基于 MyBatis 框架, 每个 Mapper 接口对应一张数据库表:

- `ChargingRequestMapper.insert(request)` 将新建的充电请求持久化到 `charging_request` 表
- `ChargingRecordMapper` 负责充电记录的读写 (开始充电时插入, 结束充电时更新 `EndTime`)
- `BillMapper` 负责账单的读写 (充电结束时生成账单)
- `ChargingPileMapper` 负责充电桩状态的读写 (启动/关闭/故障标记)

业务/应用层不直接操作数据库, 所有 SQL 操作封装在持久化层的 `Mapper` 接口中。这保证了数据访问的统一性和可维护性。

1.2.5 请求处理流程总结

综合上述层次，一次“提交充电申请”的完整处理路径为：用户界面层 → 控制器层 → 业务/应用层 → 持久化层 → SQLite，响应结果再沿原路径逐级返回。具体步骤如下：

1. 用户在用户界面层选择充电模式和电量，点击提交
2. Vue.js 用户界面层发送 HTTP POST 请求到 ChargingController
3. ChargingController 校验参数，调用 ChargingService.submitRequest()
4. ChargingService 创建 ChargingRequest 对象,调用 ChargingRequestMapper.insert() 持久化
5. ChargingService 调用 QueueService.enqueue() 将请求加入等待队列
6. QueueService 计算排队序号，返回排队信息
7. 结果沿原路径逐级返回：业务/应用层 → 控制器层 → Vue.js 用户界面层 → 用户界面展示

该流程体现了各层之间的松耦合关系：用户界面层不感知数据库结构，控制器层不处理业务逻辑，业务/应用层不直接操作 SQL，持久化层不涉及业务判断。

2 系统的界面设计

2.1 用户端界面设计

用户端围绕车辆充电全过程组织界面，侧边栏提供“充电申请、排队状态、充电过程、账单查询”四个功能入口，顶部摘要卡片显示当前车辆、申请状态、分配桩号和实时费用，便于用户在一个工作台内完成从提交申请到查看账单的完整流程。

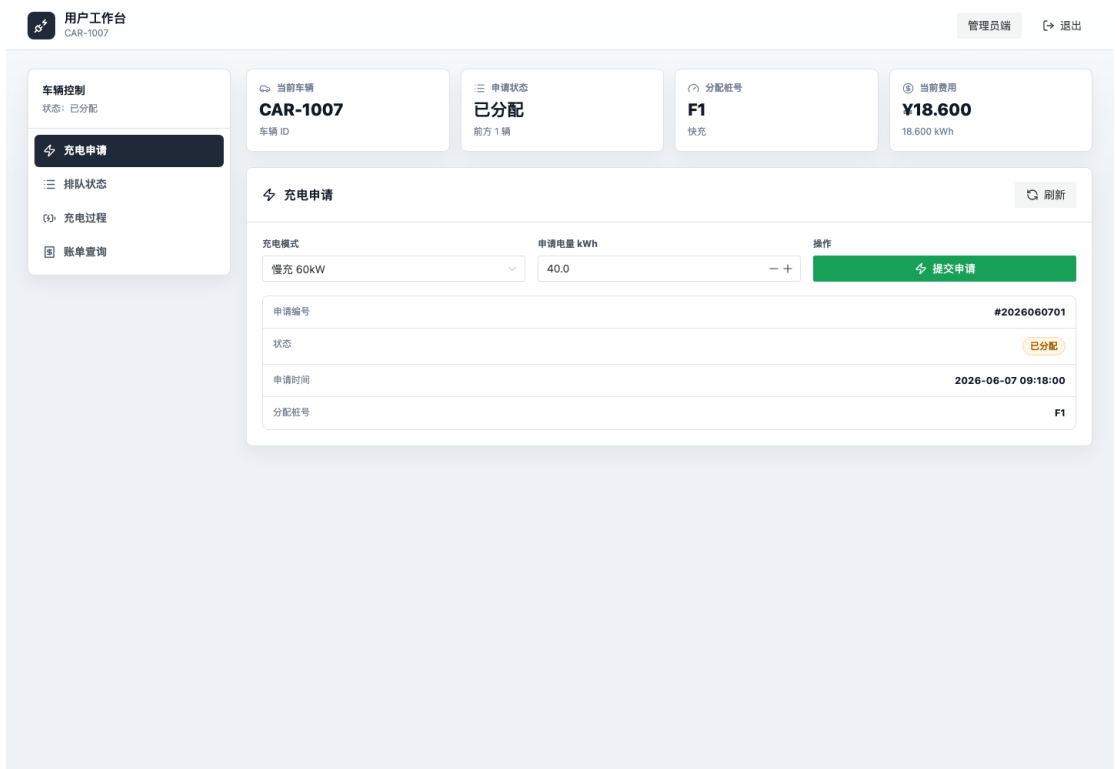


图 2：用户端充电申请界面

图2展示用户提交充电申请的界面。用户选择慢充或快充模式，输入申请电量后提交请求；界面同时保留申请编号、申请状态、申请时间和分配桩号等信息区，用于展示请求提交后的基本状态。

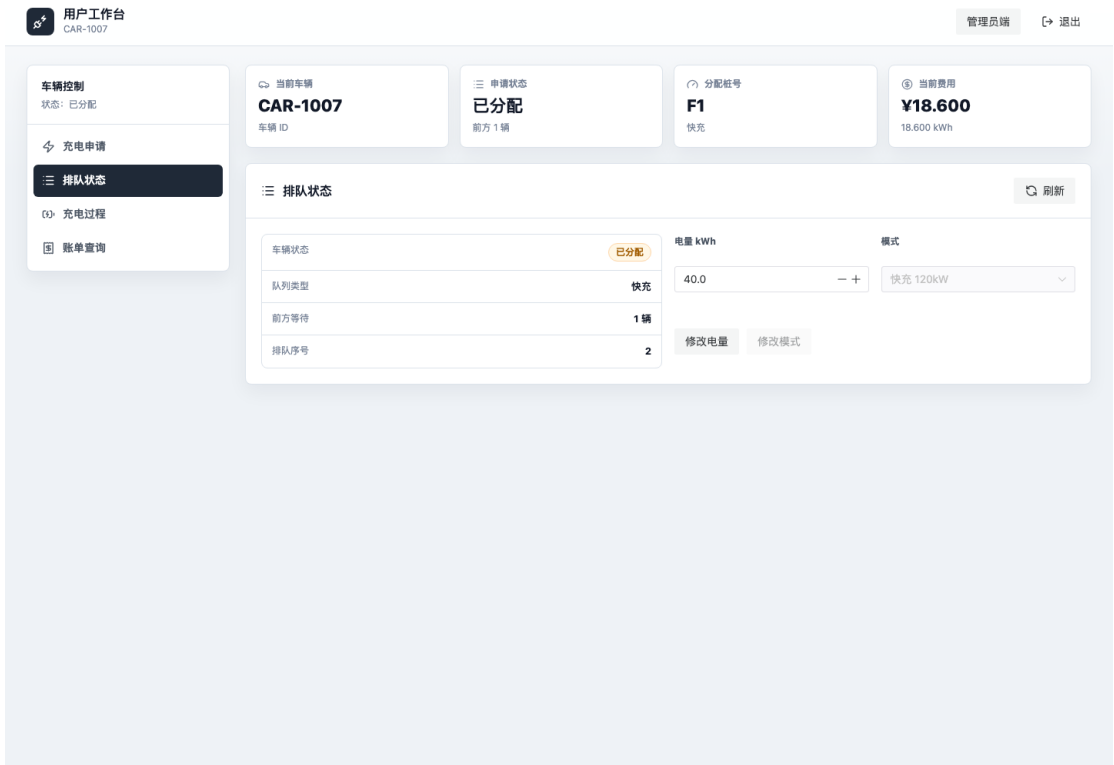


图 3: 用户端排队状态界面

图3展示用户查看排队状态的界面。该界面显示车辆状态、队列类型、前方等待车辆数、排队序号等信息，并提供修改申请电量和修改充电模式的操作入口；当车辆已被调度到充电桩时，界面会同步展示分配桩号。

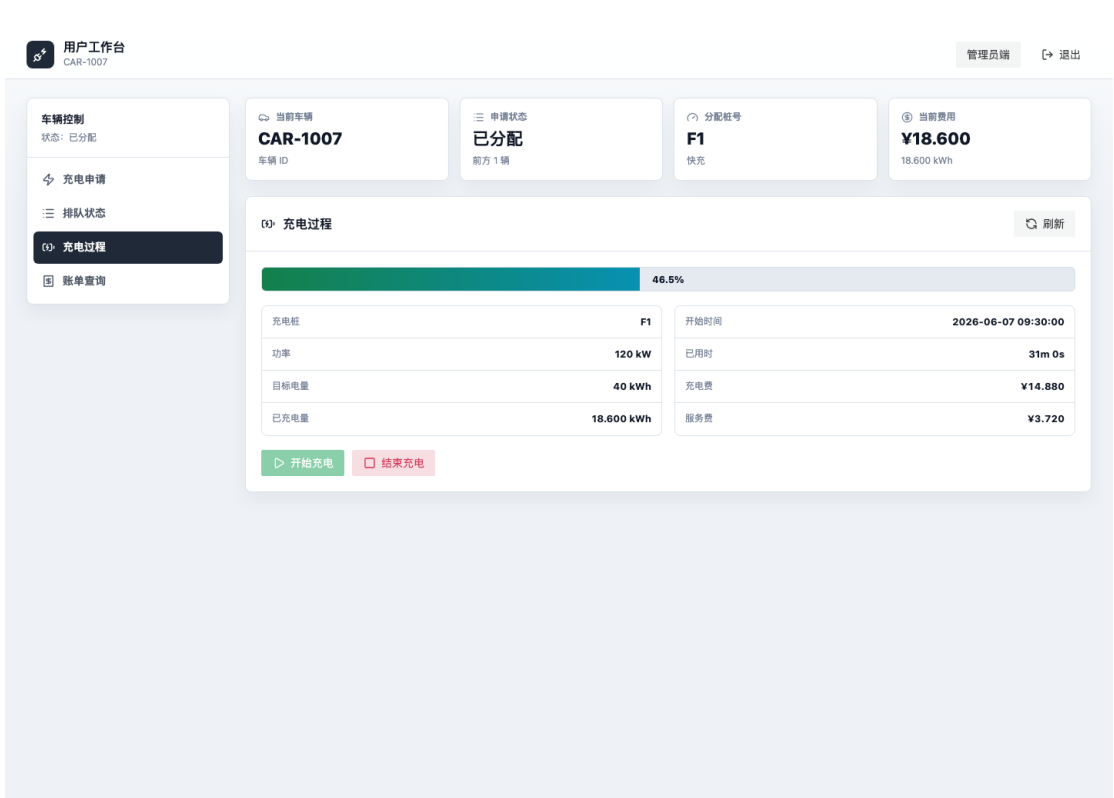


图 4：用户端充电过程监控界面

图4展示充电过程监控界面。系统以进度条展示当前充电进度，并显示充电桩编号、充电功率、目标电量、已充电量、开始时间、已用时、充电费和服务费；底部提供开始充电和结束充电按钮，对应用户端充电控制操作。

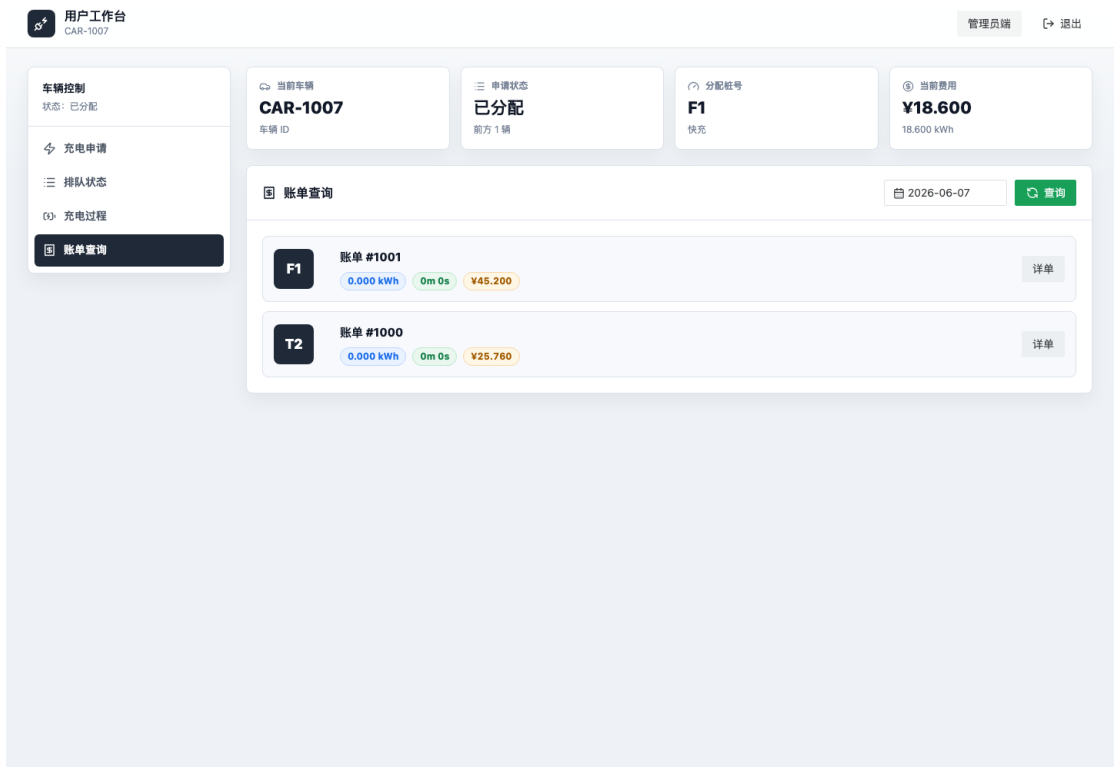


图 5：用户端账单查询界面

图5展示账单查询界面。用户可按日期查询历史账单，列表中展示账单编号、充电桩编号、充电电量、充电费用、服务费用和总费用，并可进入详单查看分时计费明细。

2.2 管理员端界面设计

管理员端采用同样的工作台布局，围绕充电站运行管理组织为“充电桩状态、等待队列、计费参数”三个功能区。顶部摘要卡片展示开机数量、累计充电次数、累计充电量和当前队列车辆数；侧边栏用于在各管理功能之间快速切换。

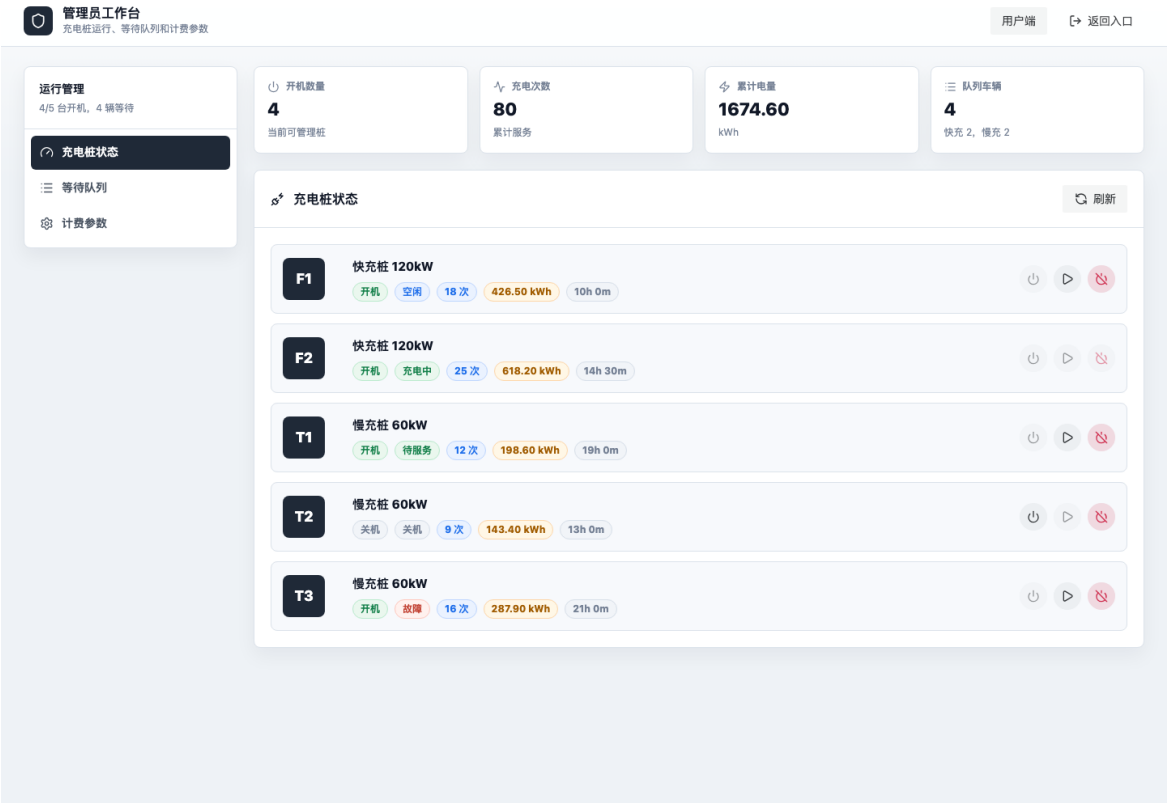


图 6: 管理员端充电桩状态管理界面

图6展示充电桩状态管理界面。界面以卡片列表展示多个充电桩，包括快充桩和慢充桩的桩号、功率类型、电源状态、运行状态、累计充电次数、累计充电量和累计运行时长；每个充电桩右侧提供开机、置为待服务、关机管理等管理按钮。示例中同时展示了空闲、充电中、待服务、关机和故障等不同状态，便于体现界面样式和状态区分。

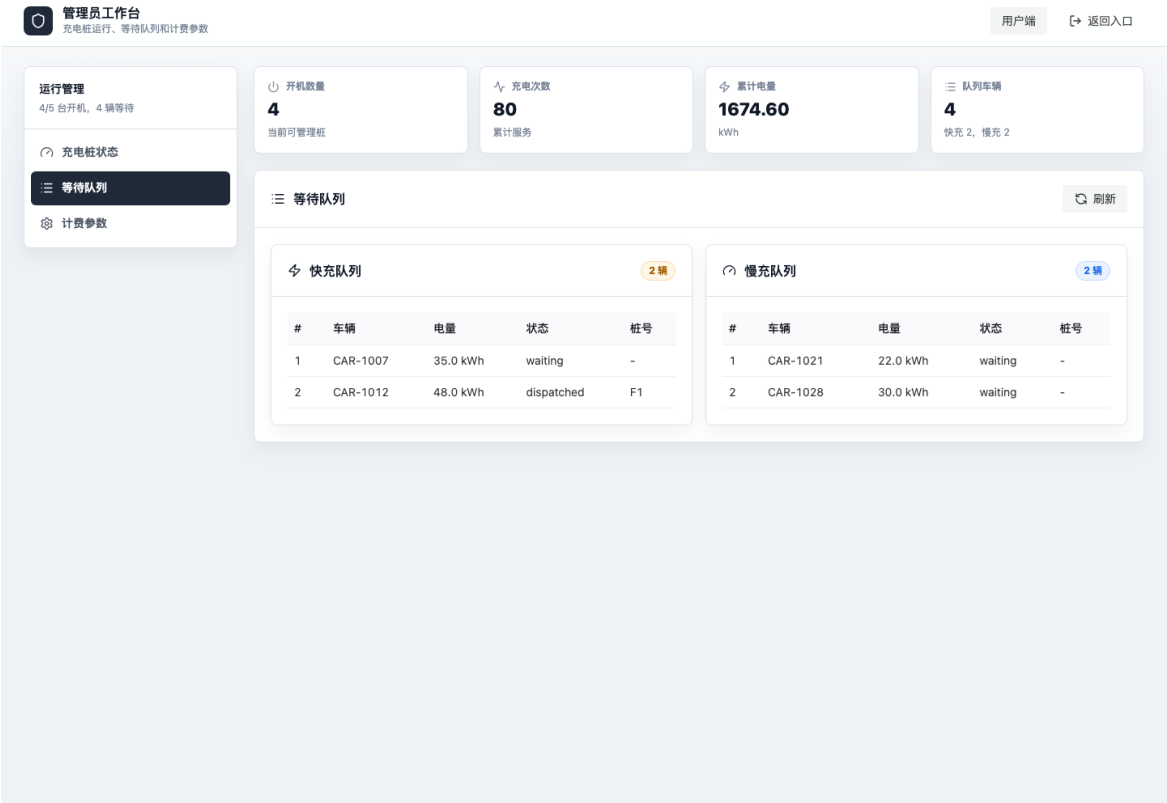


图 7：管理员端等待队列监控界面

图7展示等待队列监控界面。管理员可分别查看快充队列和慢充队列中的车辆，表格展示排队序号、车辆编号、请求电量、车辆状态和已分配桩号，用于支持调度过程中的队列状态检查。

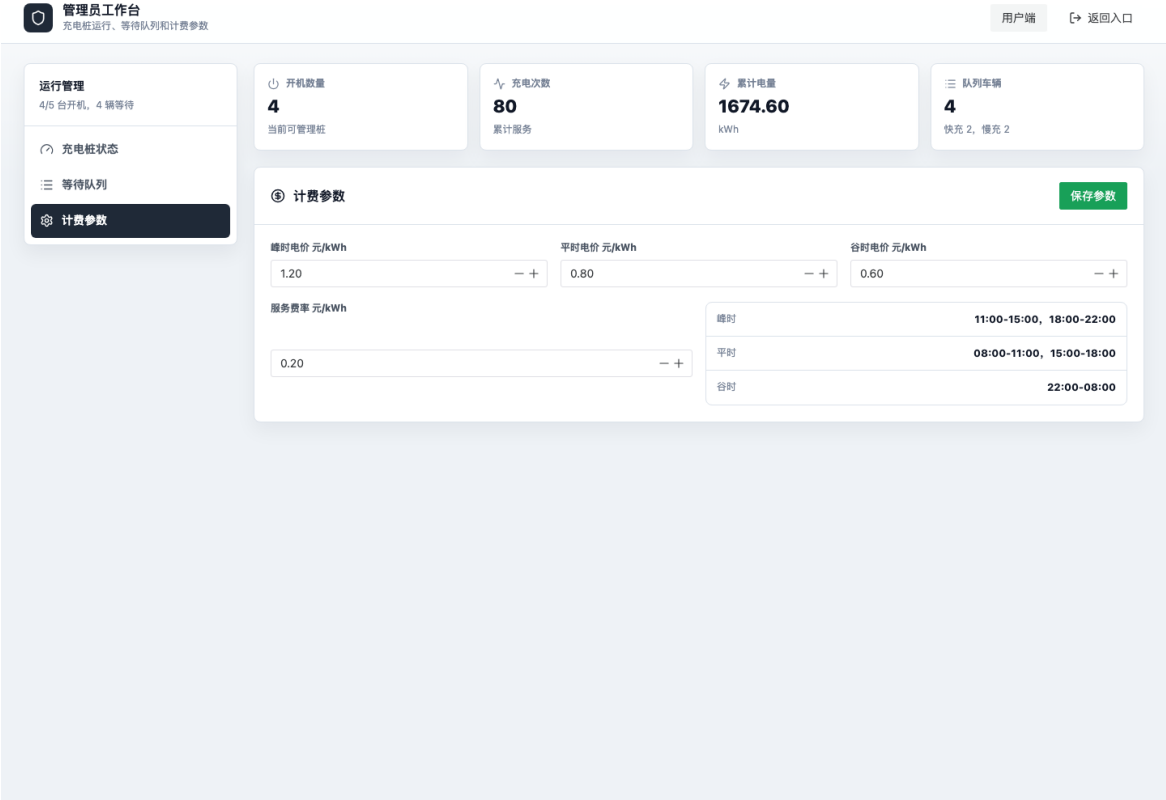


图 8: 管理员端计费参数设置界面

图8展示计费参数设置界面。管理员可维护峰时电价、平时电价、谷时电价和服务费率，界面右侧给出峰平谷时间段说明，保存后的参数用于后续账单计算。

- **用户端功能覆盖：**提交充电申请、查看排队状态、监控充电过程、查询账单和详单。
- **管理员端功能覆盖：**查看多根充电桩状态、执行充电桩启停管理、监控快充/慢充等待队列、维护计费参数。
- **界面组织方式：**两端均采用顶部状态摘要、左侧导航、右侧卡片式内容区的布局，提高功能切换和状态查看效率。

3 系统动态结构设计

3.1 UC1 注册

3.1.1 已知条件

表 1: UC1 注册系统事件表

消息名称	参数列表	返回值
createNewAccount	car_Id, userName, car_Capacity	0: 成功; 1: 失败
set_pwd	password	0: 成功; 1: 失败

3.1.2 对象设计: createNewAccount(car_Id, userName, car_Capacity)

操作契约:

系统事件	createNewAccount(car_Id, userName, car_Capacity)
交叉引用	注册
前置条件	用户首次使用系统, car_Id 未被注册
后置条件	1. 一个新的 User 对象实例被创建 2. User 对象的 car_Id 属性设置为 car_Id, userName 属性设置为 userName, car_Capacity 属性设置为 car_Capacity, state 属性设置为“未激活” 3. User 对象与系统建立关联

对象设计说明:

用例角色 User 通过 UserClient 前端发起注册请求。UserController 接收该请求后调用 UserService.createAccount() 方法。UserService 首先校验 car_Id 的唯一性, 然后创建 User 业务对象并初始化其属性 (car_Id、userName、car_Capacity、state=“未激活”), 最后通过 UserMapper.insert() 将 User 对象持久化到数据库。建立的关联关系: User 与数据库之间建立持久化关联。需要初始化 User 对象的全部属性。

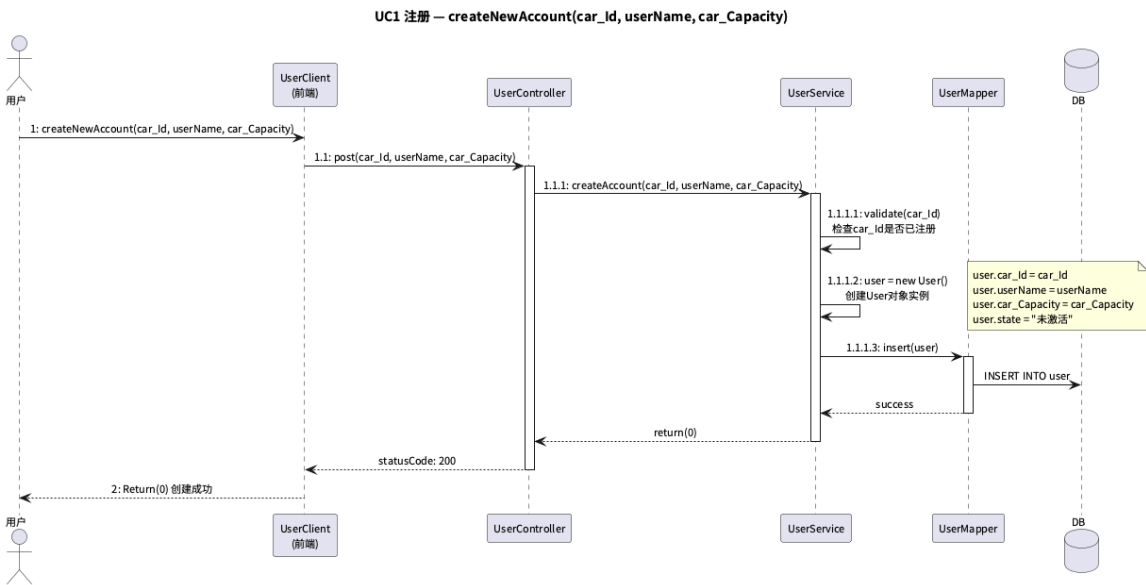


图 9: createNewAccount(car_Id, userName, car_Capacity) 交互图

3.1.3 对象设计：set_pwd(password)

操作契约：

系统事件	set_pwd(password)
交叉引用	注册
前置条件	createNewAccount 已成功执行，User 对象存在且状态为“未激活”
后置条件	1. User 对象的 password 属性被设置为加密后的密码 2. User 对象的 state 属性被修改为“已激活”

对象设计说明：

UserController 接收 set_pwd 请求后调用 UserService.setPassword() 方法。UserService 通过 UserMapper.findById() 查询 User 对象，对 password 进行加密处理后调用 User 对象的 setPassword() 和 setState() 方法修改属性，最后通过 UserMapper.update() 将修改持久化到数据库。

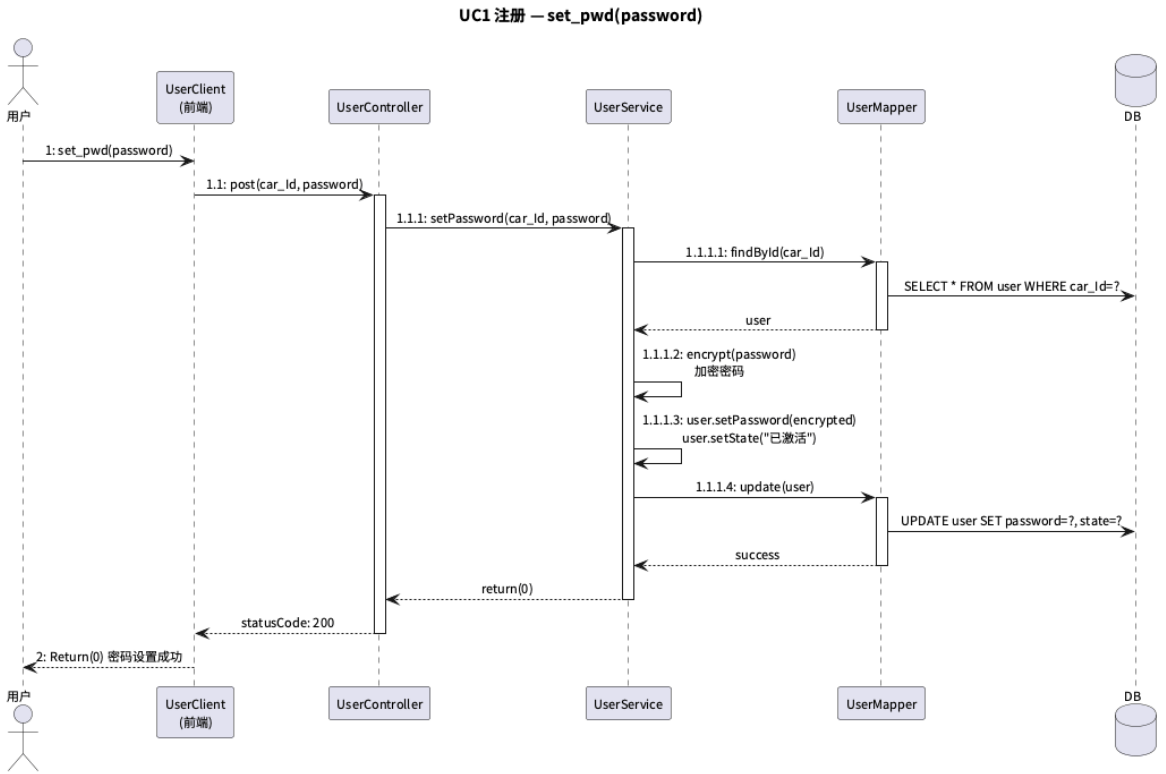


图 10: set_pwd(password) 交互图

3.2 UC2 充电申请

3.2.1 已知条件

表 2: UC2 充电申请系统事件表

消息名称	参数列表	返回值
E_charging Request	car_Id, Request_Amount, Request_Mode	car_position, car_state, queue_Num, request_time
Modify Amount	car_Id, Amount	0: 成功; 1: 失败
Modify Mode	car_Id, Mode	0: 成功; 1: 失败
Query_Car State	car_id	car_Number_before_position, car_state, queue_Num, request_time
Start Charging	car_id, ChargePileNum	0: 成功; 1: 失败
Query_Charging State	car_id	详单信息（具体参数详见需求说明）
End Charging	car_id, ChargingPileNum	0: 成功; 1: 失败

3.2.2 对象设计:E_chargingRequest(car_Id, Request_Amount, Request_Mode)

操作契约:

系统事件	E_chargingRequest(car_Id, Request_Amount, Request_Mode)
交叉引用	充电申请
前置条件	用户已登录, 车辆在充电站等候区
后置条件	1. 一个新的 ChargingRequest 对象实例被创建 2. ChargingRequest 对象的 car_Id 属性设置为 car_Id, Request_Amount 属性设置为 Request_Amount, Request_Mode 属性设置为 Request_Mode, request_time 属性 设置为系统当前时间, car_state 属性设置为 “等待” 3. ChargingRequest 对象与 User 对象建立关联 4. ChargingRequest 对象与对应充电模式的等待队列对象建立 关联

对象设计说明:

ChargingController 接收请求后调用 ChargingService.submitRequest() 方法。ChargingService 创建 ChargingRequest 对象并初始化属性 (car_Id、Request_Amount、Request_Mode、request_time=now()、car_state= “等待”), 通过 ChargingRequestMapper.insert() 持久化, 然后调用 QueueService.enqueue() 将请求加入对应充电模式的

等待队列，QueueService 计算并返回排队序号。

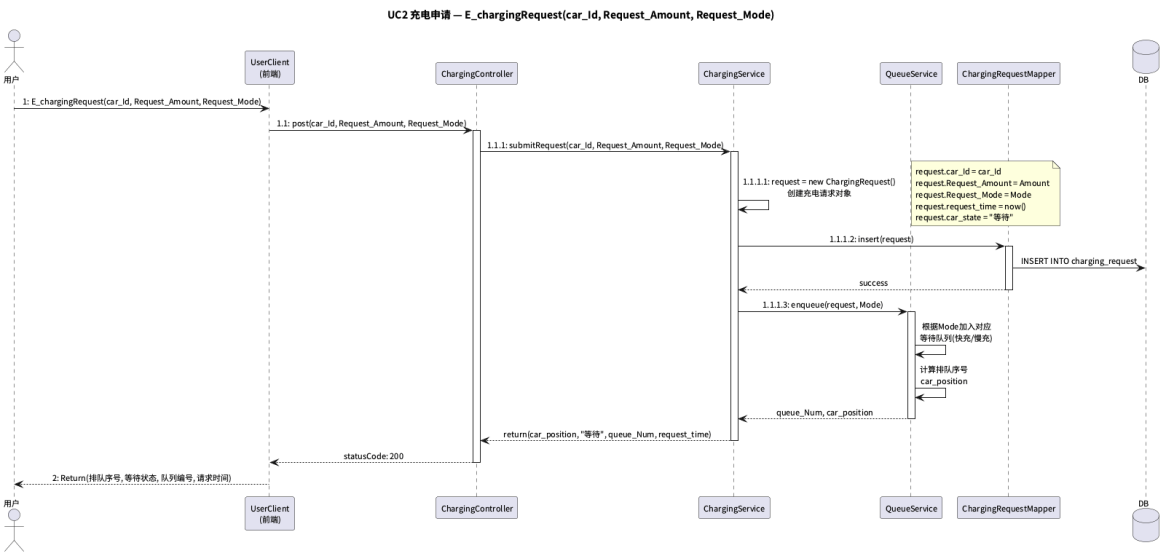


图 11: E_chargingRequest 交互图

3.2.3 对象设计: Modify_Amount(car_Id, Amount)

操作契约:

系统事件	Modify_Amount(car_Id, Amount)
交叉引用	充电申请
前置条件	已提交请求且处于等待状态
后置条件	ChargingRequest 对象的 Request_Amount 属性被修改为 Amount

对象设计说明:

ChargingService 通过 ChargingRequestMapper.findByCarId() 查询 ChargingRequest 对象，调用其 setRequest_Amount() 修改属性，再通过 Mapper.update() 持久化。由于仅修改充电电量不影响排队顺序，无需调整队列。

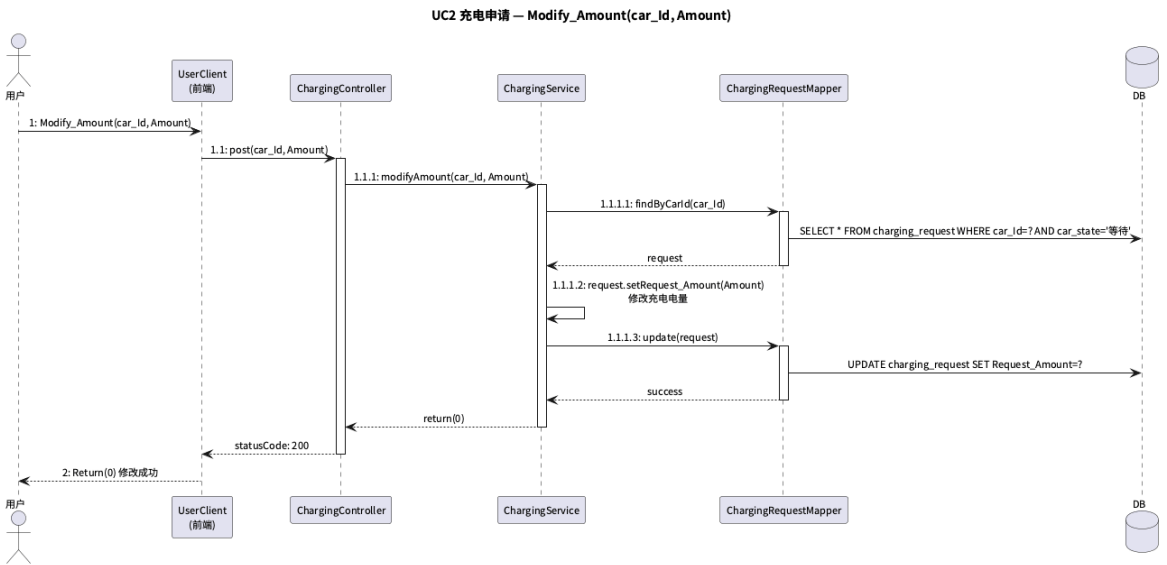


图 12: Modify_Amount 交互图

3.2.4 对象设计：Modify_Mode(car_Id, Mode)

操作契约：

系统事件	Modify_Mode(car_Id, Mode)
交叉引用	充电申请
前置条件	已提交请求且处于等待状态
后置条件	1. ChargingRequest 对象的 Request_Mode 属性被修改为 Mode 2. ChargingRequest 对象与原等待队列的关联被取消，与新等待队列建立关联 3. 原等待队列和新等待队列的排队序号属性被重新计算

对象设计说明：

Modify
与Amount不同，修改充电模式需要跨队列操作。ChargingService 先调用 QueueService.dequeue() 从原等待队列移除该请求，然后修改 Request_Mode 属性，再调用 QueueService.enqueue() 以原始 request_time 加入新模式队列。由于充电模式变化，可能影响该请求的调度优先级。

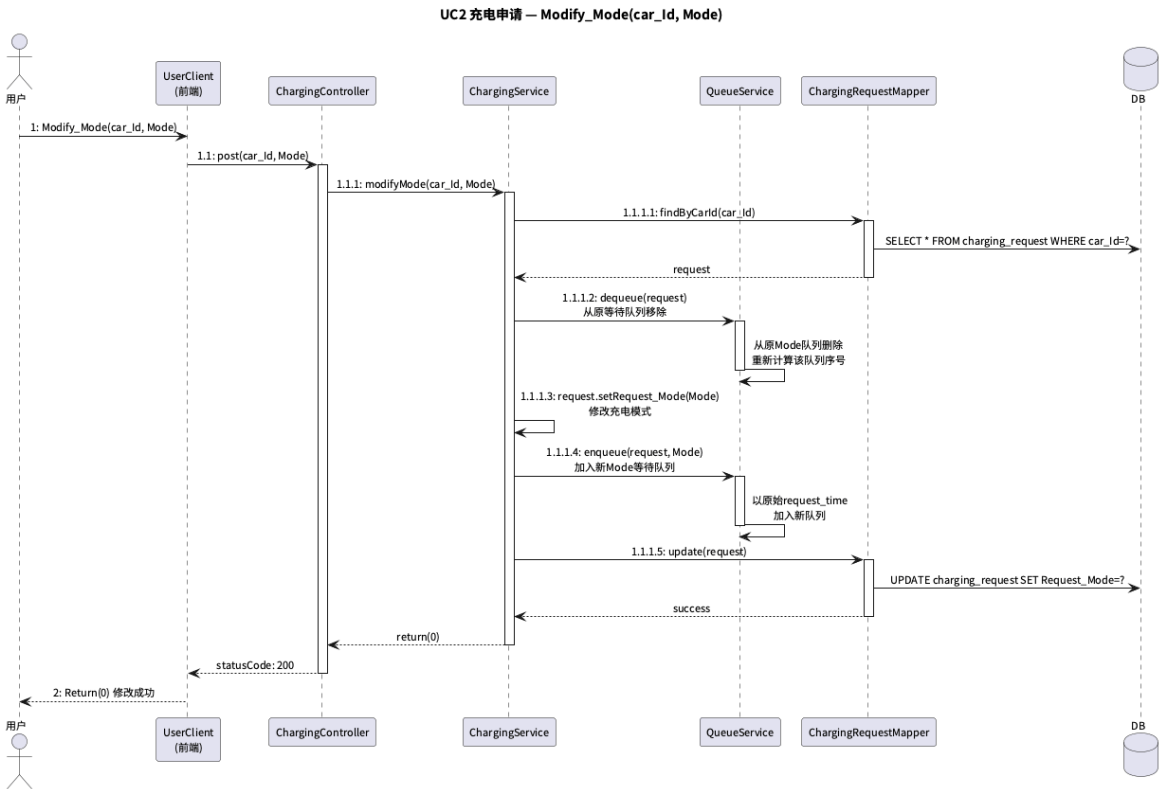


图 13: Modify_Mode 交互图

3.2.5 对象设计: Query_Car_State(car_id)

操作契约:

系统事件	Query_Car_State(car_id)
交叉引用	充电申请
前置条件	已提交充电请求
后置条件	无

对象设计说明:

纯查询操作。ChargingService 先通过 Mapper 查询 ChargingRequest 对象, 取得车辆状态(car_state)和请求时间(request_time), 再调用 QueueService.getPosition() 计算前方等待车辆数(car_Number_before_position)和队列编号(queue_Num), 最后返回上述四项信息。无对象创建、修改或删除。

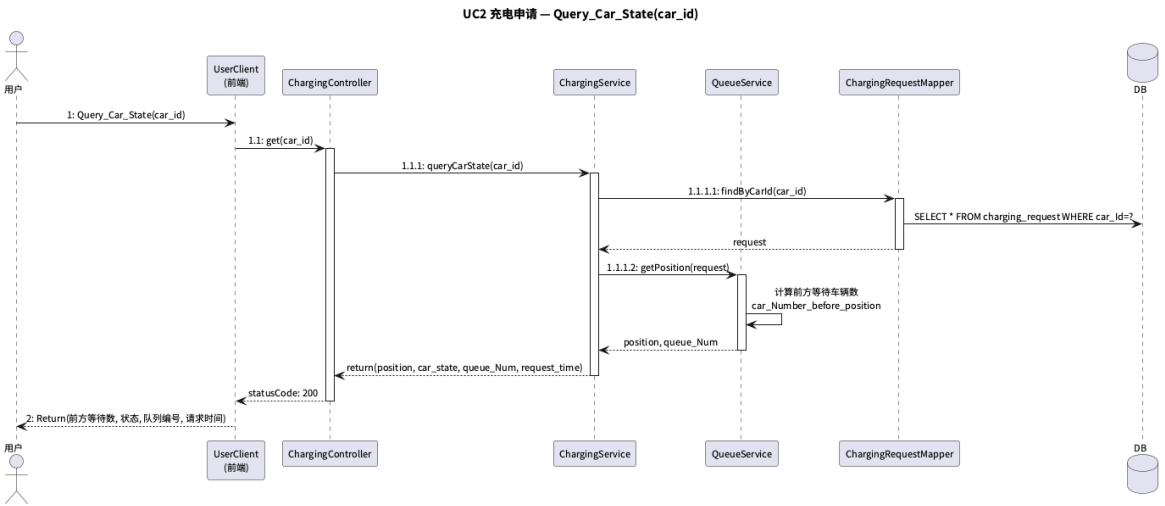


图 14: Query_Car_State 交互图

3.2.6 对象设计：Start_Charging(car_id, ChargePileNum)

操作契约：

系统事件	Start_Charging(car_id, ChargePileNum)
交叉引用	充电申请
前置条件	系统已将充电桩分配给该车辆，车辆到达充电桩，且 ChargingRequest.Request_Mode 与 ChargingPile.type 一致
后置条件	1. ChargingRequest 对象与 ChargingPile 对象的充电类型匹配校验通过 2. 一个新的 ChargingRecord 对象实例被创建 3. ChargingRecord 对象的 car_id 属性设置为 car_id, ChargePileNum 属性设置为 ChargePileNum, StartTime 属性设置为系统当前时间 4. ChargingRecord 对象与 ChargingRequest 对象建立关联 5. ChargingPile 对象的 workingState 属性被修改为“充电中” 6. ChargingRequest 对象与等待队列的关联被取消，与充电桩服务队列建立关联

对象设计说明：

ChargingService 开始充电前先查询 ChargingRequest 和 ChargingPile 对象,校验 ChargingRequest.Request_Mode 与 ChargingPile.type 是否一致；若类型不匹配，则返回失败，不创建充电记录。校验通过后，ChargingService 创建 ChargingRecord 对象(car_id、ChargePileNum、StartTime=now())，通过 ChargingRecordMapper.insert() 持久化。然后调用 PileService.updatePileState() 将充电桩状态更新为“充电中”，调用 QueueService.dequeue() 将该请求从等待队列移除并加入对应充电桩的服务队列。

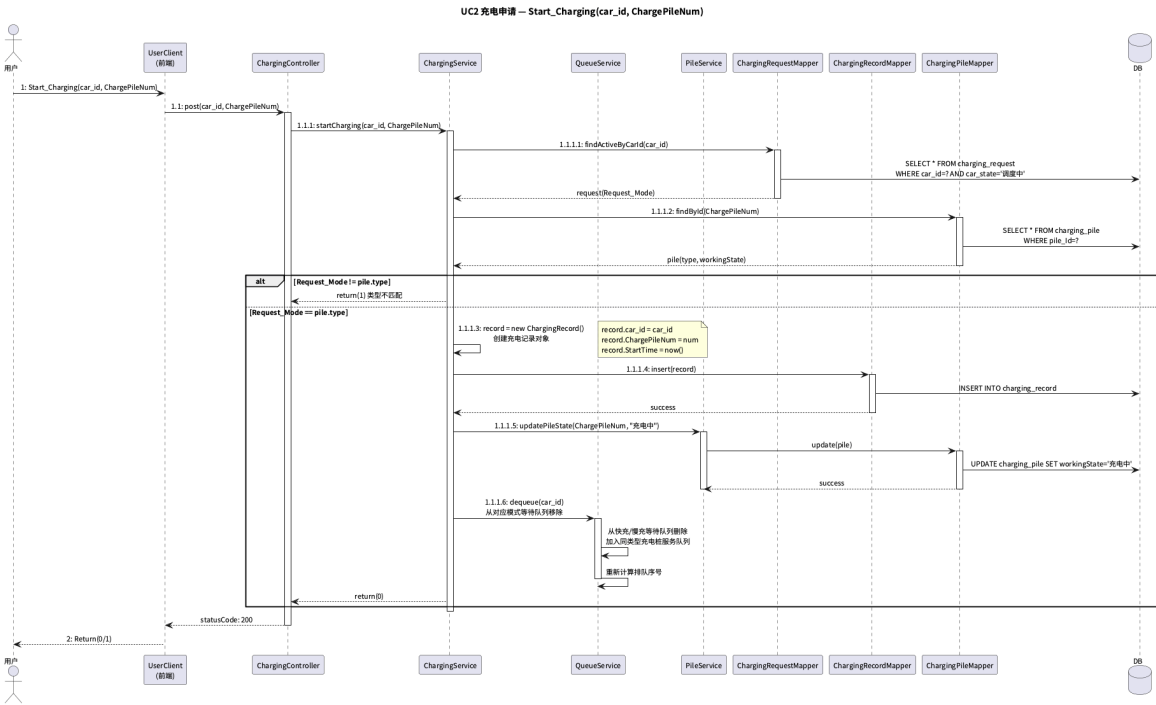


图 15: Start_Charging 交互图

3.2.7 对象设计：Query_Charging_State(car_id)

操作契约：

系统事件	Query_Charging_State(car_id)
交叉引用	充电申请
前置条件	用户正在充电
后置条件	无

对象设计说明：

ChargingService 通过 Mapper 查询正在进行的 ChargingRecord，计算已充电时长 (now() - StartTime) 和已充电量 (时长 × 功率)，调用 BillingService.calculateFee() 根据时段电价和服务费率实时计算当前费用，最后组装并返回充电状态详单信息。该详单信息用于展示车辆当前充电状态、已充电时长、已充电量和当前费用等充电过程数据。

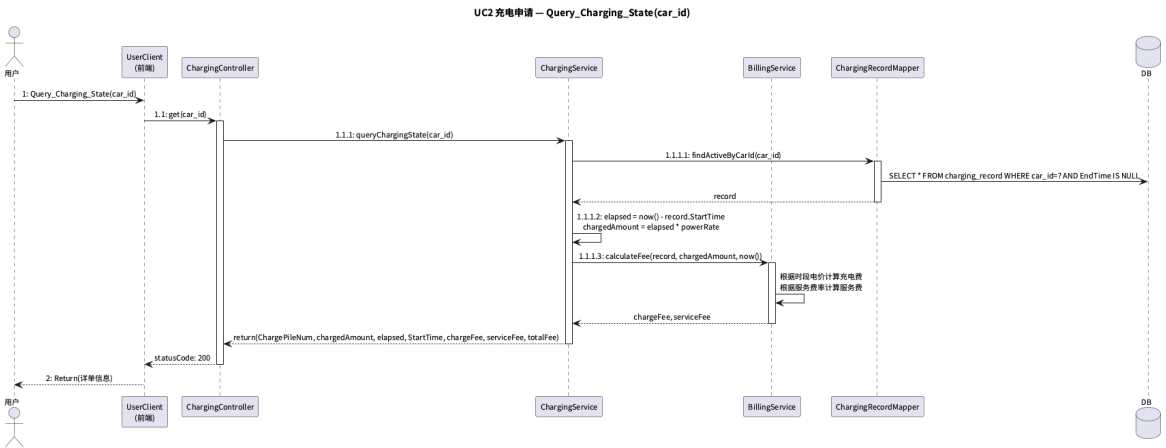


图 16: Query_Charging_State 交互图

3.2.8 对象设计：End_Charging(car_id, ChargingPileNum)

操作契约：

系统事件	End_Charging(car_id, ChargingPileNum)
交叉引用	充电申请
前置条件	该车辆正在充电桩上充电
后置条件	1. ChargingRecord 对象的 EndTime 属性被设置为系统当前时间，ChargeAmount 属性被设置为实际充电量 2. 一个新的 Bill 对象实例被创建 3. Bill 对象的属性被初始化（包括 TotalChargeFee、TotalServiceFee、TotalFee） 4. Bill 对象与 ChargingRecord 对象建立关联 5. ChargingPile 对象的 workingState 属性被修改为“空闲”

对象设计说明：

ChargingService 查询活跃的 ChargingRecord, 设置 EndTime 和 ChargeAmount, 通过 Mapper 更新。然后调用 BillingService.generateBill() 创建 Bill 对象（汇总计算 TotalChargeFee 和 TotalServiceFee）并通过 BillMapper 持久化。接着调用 PileService.updatePileState() 将充电桩状态恢复为“空闲”,最后调用 QueueService.dispatchNext() 触发调度策略从等待队列中选取下一辆车分配充电桩。

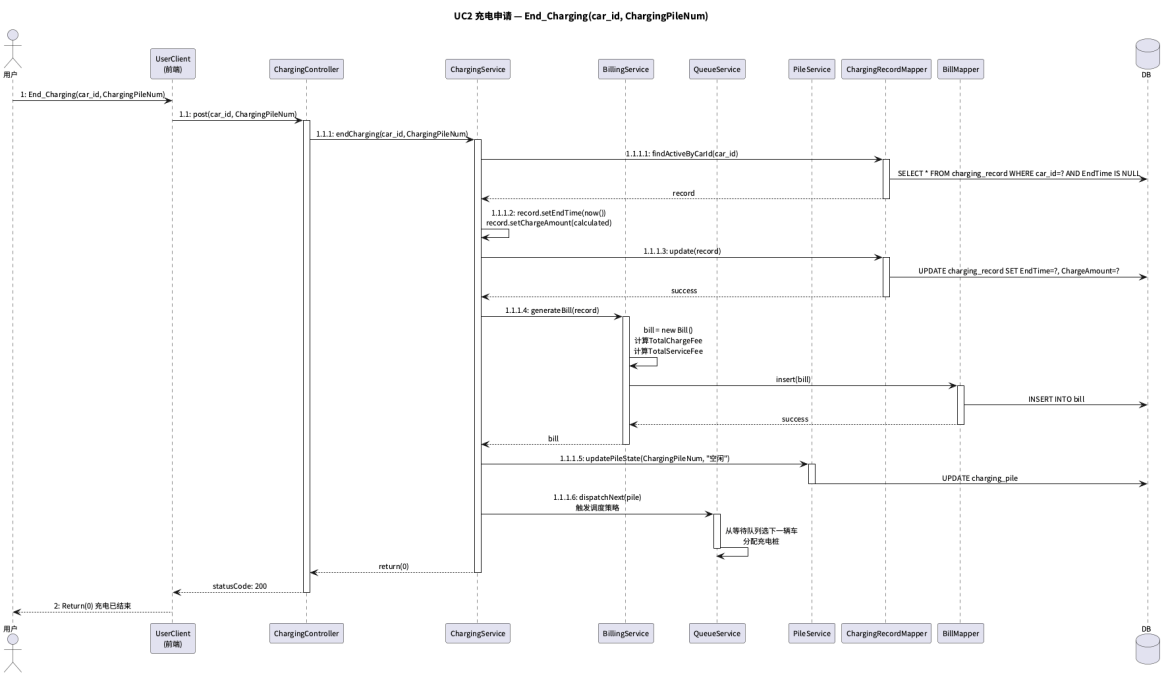


图 17: End_Charging 交互图

3.3 UC3 查看账单

3.3.1 已知条件

表 3: UC3 查看账单系统事件表

消息名称	参数列表	返回值
Request Bill	carId, date	carId, date, Bill_Id, ChargePileNum, ChargeAmount, ChargeDuration, StartTime, EndTime, TotalChargeFee, TotalServiceFee, TotalFee

3.3.2 对象设计：Request_Bill(carId, date)

操作契约：

系统事件	Request_Bill(carId, date)
交叉引用	UC3 查看账单
前置条件	用户已登录， 存在该日期范围的充电记录或账单记录
后置条件	无

对象设计说明：

BillController 接收请求后调用 BillService.listBills() 方法。BillService 通过 BillMapper 查询该车辆在指定日期的账单列表，并返回 carId、date、Bill_Id、ChargePileNum、ChargeAmount、ChargeDuration、StartTime、EndTime、TotalChargeFee、TotalServiceFee 和 TotalFee 等账单信息。查看账单属于查询操作，不创建或修改业务对象。

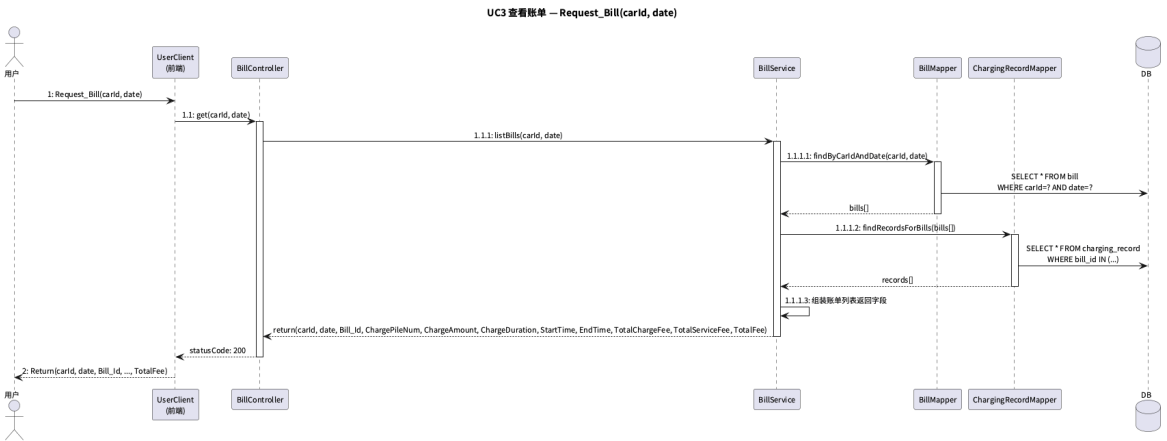


图 18: Request_Bill 交互图

3.4 UC4 查看详情单

查看详情单是用户在已获得账单编号后进一步查看账单明细的独立用例。它依赖 UC3 查看账单返回的 Bill_Id，但其用户目标、输入参数和返回信息均相对独立，因此单独编号为 UC4。

3.4.1 已知条件

表 4：UC4 查看详情单系统事件表

消息名称	参数列表	返回值
Request_Detailed List	Bill_Id	carId, date, Bill_Id, ChargePileNum, ChargeAmount, ChargeDuration, StartTime, EndTime, ChargeFee, ServiceFee, subtotalFee

3.4.2 对象设计：Request_DetailedList(Bill_Id)

操作契约：

系统事件	Request_DetailedList(Bill_Id)
交叉引用	UC4 查看详情单
前置条件	用户已登录，账单 Bill_Id 已存在
后置条件	无

对象设计说明：

BillController 接收请求后调用 BillService.getBillDetail() 方法。BillService 通过 BillMapper.findById() 获取 Bill 对象，再通过 ChargingRecordMapper.findById() 查询该账单关联的充电记录，组装Response 详单信息（包含 carId、date、Bill_Id、ChargePileNum、ChargeAmount、ChargeDuration、StartTime、EndTime、ChargeFee、ServiceFee 和 subtotalFee 等字段）。查看详情单属于查询操作，不创建或修改业务对象。

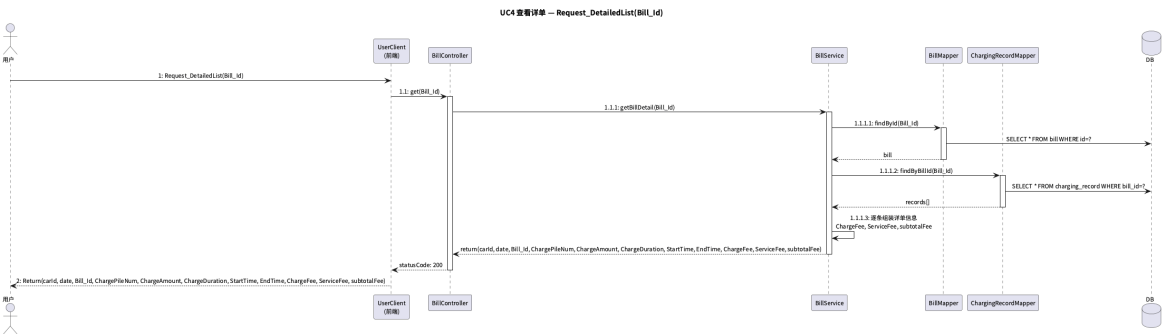


图 19：Request_DetailedList 交互图

3.5 UC5 运行充电桩

3.5.1 已知条件

表 5: UC5 运行充电桩系统事件表

消息名称	参数列表	返回值
powerOn	pile_Id	0: 成功; 1: 失败
setParameters	计费规则、电价数据	0: 成功; 1: 失败
Start Charging Pile	pile_Id	0: 成功; 1: 失败
powerOff	pile_Id	0: 成功; 1: 失败

3.5.2 对象设计: powerOn(pile_Id)

操作契约:

系统事件	powerOn(pile_Id)
交叉引用	UC5 运行充电桩
前置条件	管理员已登录, 充电桩处于关闭状态
后置条件	ChargingPile 对象的 powerState 属性被修改为“已开机”

对象设计说明:

AdminController 接收请求后调用 PileService.powerOn() 方法。PileService 通过 ChargingPileMapper.findById() 查询充电桩对象, 调用 setPowerState(“已开机”) 修改属性, 再通过 Mapper.update() 持久化。

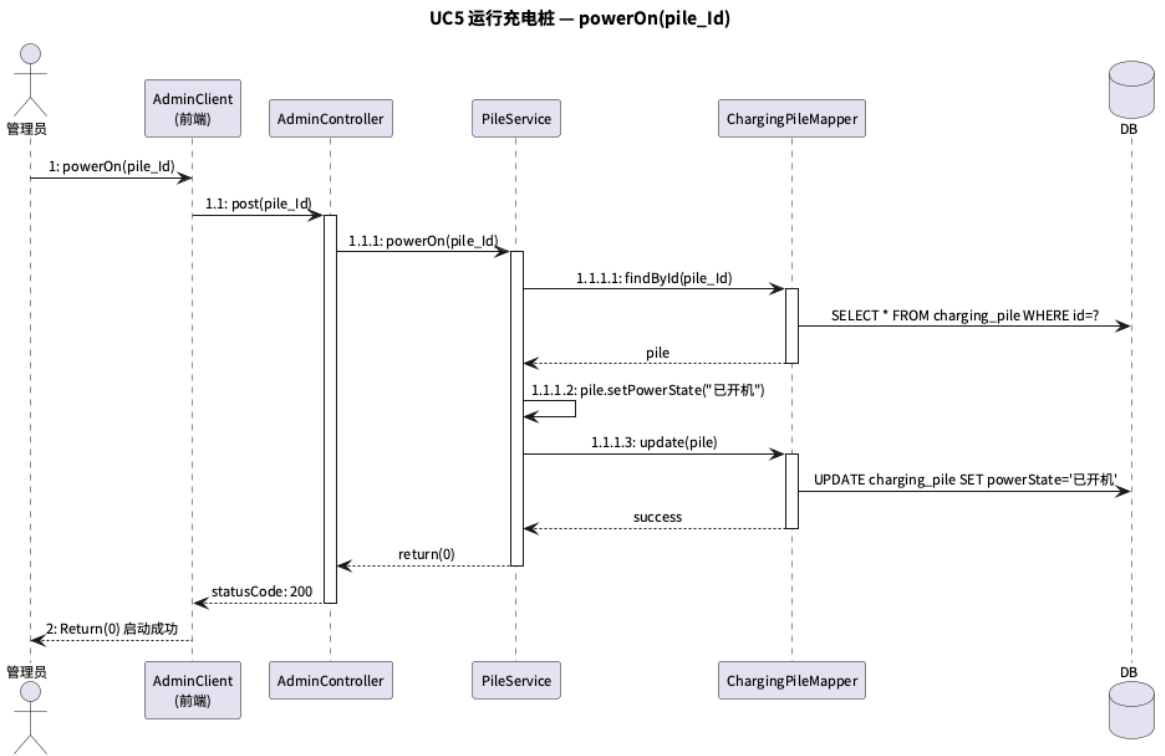


图 20: powerOn 交互图

3.5.3 对象设计：setParameters(peakPrice, normalPrice, valleyPrice, serviceFeeRate)

操作契约：

系统事件	setParameters(peakPrice, normalPrice, valleyPrice, serviceFeeRate)
交叉引用	UC5 运行充电桩
前置条件	管理员已登录
后置条件	1. 一个新的 PricingConfig 对象实例被创建（若不存在） 2. PricingConfig 对象的 peakPrice 属性设置为 peakPrice, normalPrice 属性设置为 normalPrice, valleyPrice 属性设置为 valleyPrice, serviceFeeRate 属性设置为 serviceFeeRate

对象设计说明：

AdminController 接收请求后调用 PricingService.setParameters() 方法。PricingService 创建或更新 PricingConfig 对象（包含 peakPrice、normalPrice、valleyPrice、serviceFeeRate 四个属性），通过Mapper .update() 持久化。新参数保存后立即对后续充电计费生效。

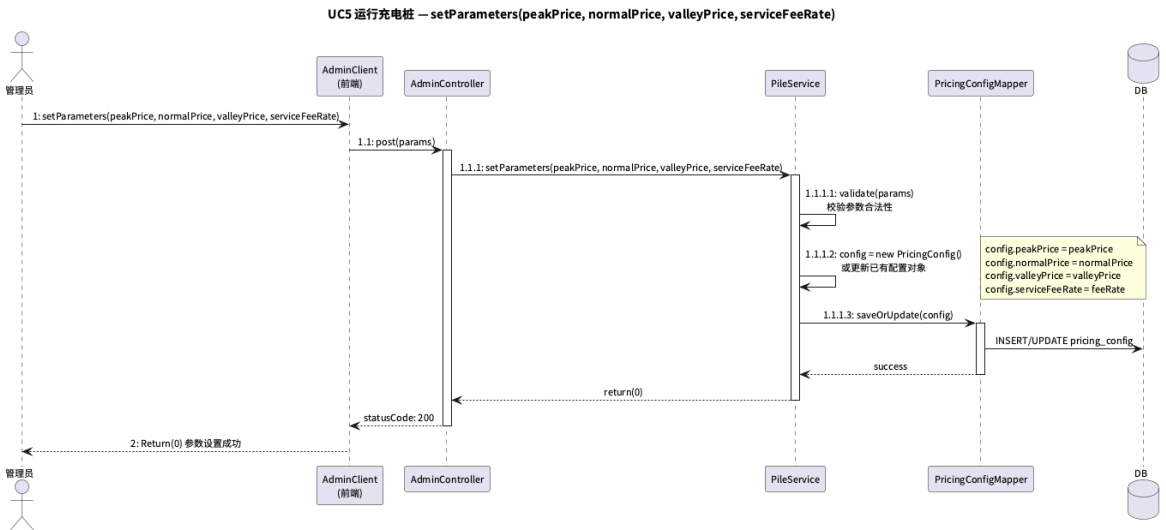


图 21: setParameters 交互图

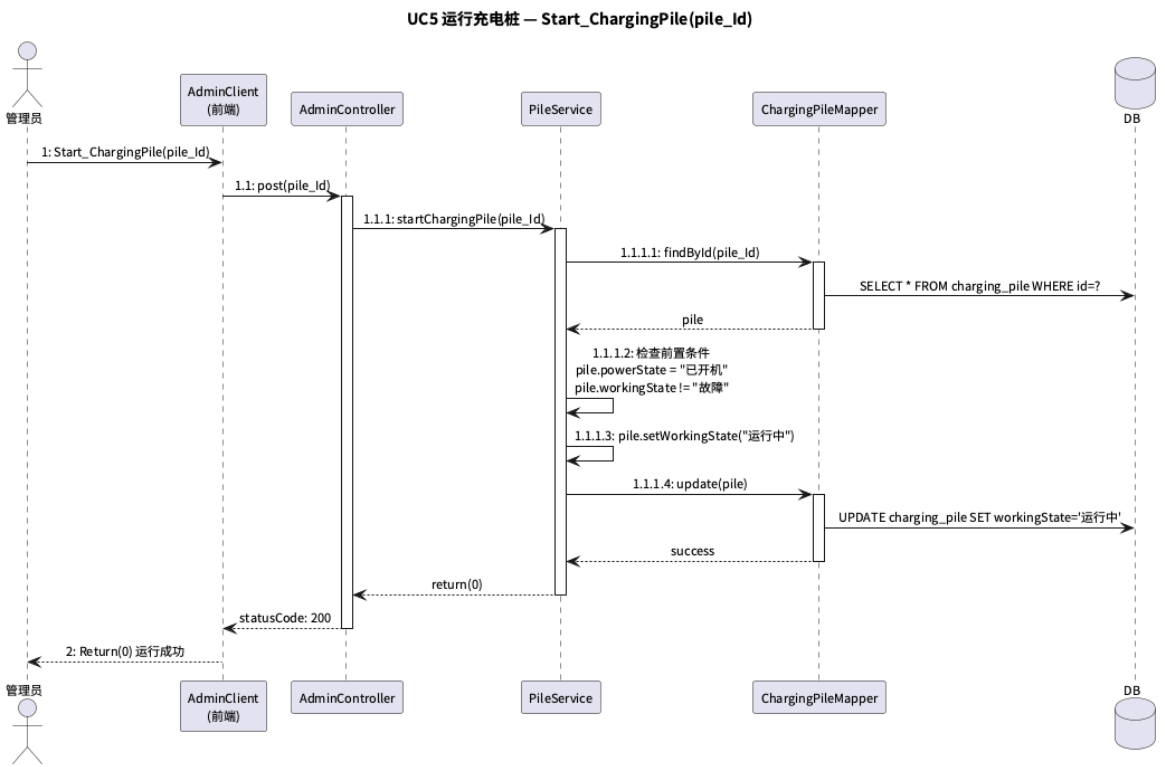
3.5.4 对象设计：Start_ChargingPile(pile_Id)

操作契约：

系统事件	Start_Charging Pile (pile_Id)
交叉引用	UC5 运行充电桩
前置条件	充电桩已开机（powerOn 已执行），当前无故障
后置条件	ChargingPile 对象的 workingState 属性被修改为“运行中”

对象设计说明：

PileService 查询充电桩，校验其 powerState= “已开机” 且 workingState 不等于 “故障”，然后调用 setWorkingState(“运行中”) 并通过 Mapper 更新。



Start_Charging
图 22: Pile 交互图

3.5.5 对象设计：powerOff(pile_Id)

操作契约：

系统事件	powerOff(pile_Id)
交叉引用	UC5 运行充电桩
前置条件	管理员已登录，充电桩无正在充电的车辆
后置条件	1. ChargingPile 对象的 powerState 属性被修改为“已关机” 2. ChargingPile 对象的 workingState 属性被修改为“已关闭”

对象设计说明：

PileService 查询充电桩，校验无正在充电的车辆（workingState 不等于“充电中”），然后同时设置 powerState=“已关机”和 workingState=“已关闭”，通过 Mapper 更新。

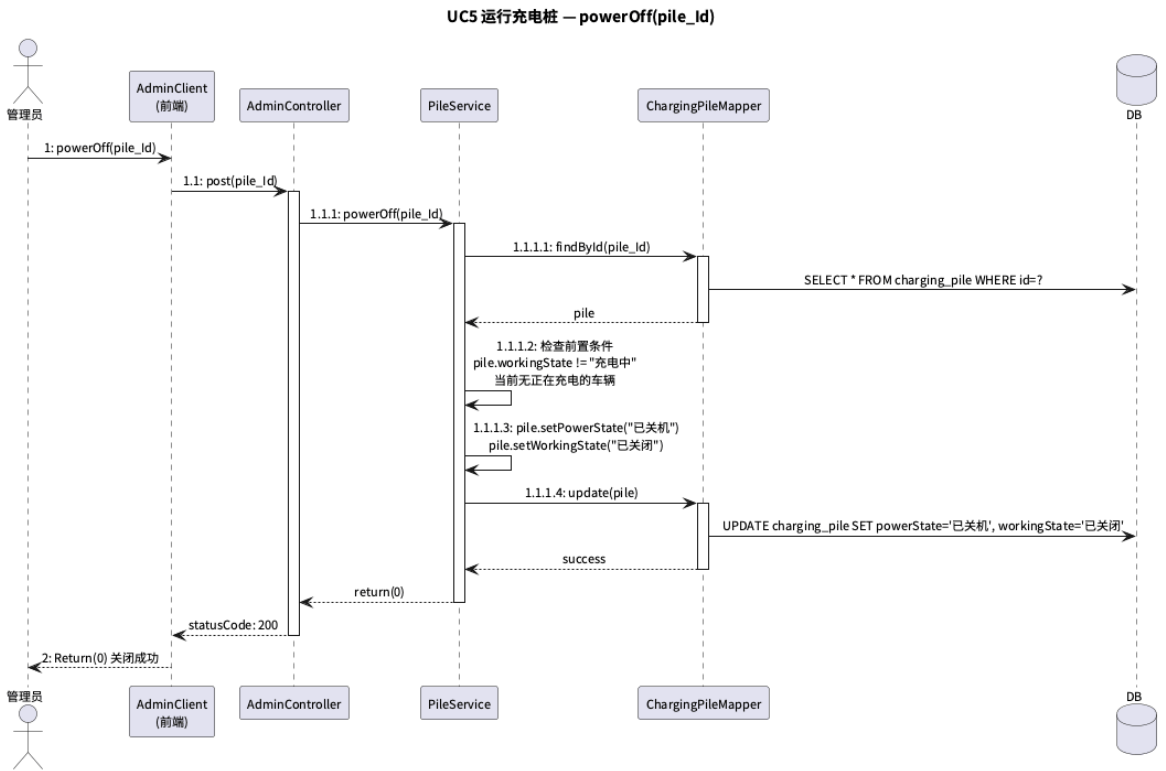


图 23: powerOff 交互图

3.6 UC6 查看充电桩状态

3.6.1 已知条件

表 6: UC6 查看充电桩状态系统事件表

消息名称	参数列表	返回值
Query_PileState	pile_Id (可空)	workingState, TotalChargeNum, TotalChargeTime, TotalCapacity

3.6.2 对象设计：Query_PileState(pile_Id)

操作契约：

系统事件	Query_PileState(pile_Id)
交叉引用	UC6 查看充电桩状态
前置条件	管理员已登录
后置条件	无

对象设计说明：

AdminController 调用 AdminMonitorService.queryPileState() 方法。AdminMonitorService 进一步调用 PileService.State()，由 PileService 通过 ChargingPileMapper 查询指定充电桩（或全部）的状态数据，包括 workingState（运行状态）、TotalChargeNum（累计充电次数）、TotalChargeTime（累计充电时长）、TotalCapacity（累计充电量）。系统支持定时刷新机制，确保状态数据实时更新。

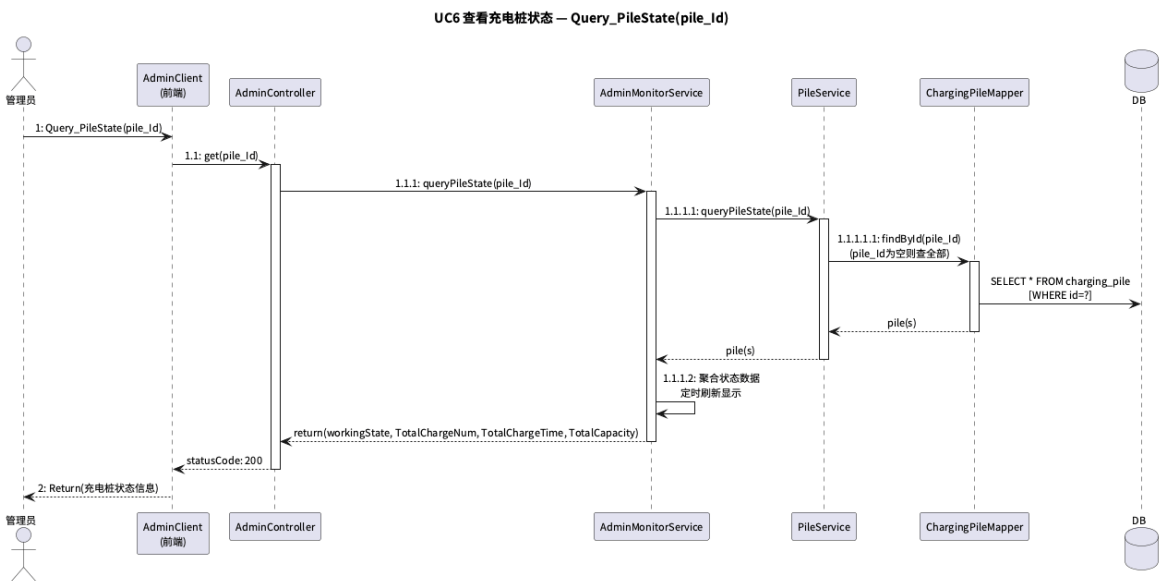


图 24: Query_PileState 交互图

3.7 UC7 查看队列状态

查看队列状态是管理员独立查看快充/慢充等待队列的管理用例。虽然它通常出现在管理员监控界面中，但其关注对象是等待队列而非单个充电桩，因此单独编号为UC7。

3.7.1 已知条件

表 7: UC7 查看队列状态系统事件表

消息名称	参数列表	返回值
Query QueueState	queuelist	car_Id, car_Capacity, Request_Amount, waitTime

3.7.2 对象设计: Query_QueueState(queuelist)

操作契约:

系统事件	Query_QueueState(queuelist)
交叉引用	UC7 查看队列状态
前置条件	管理员已登录
后置条件	无

对象设计说明:

AdminController 调用 AdminMonitorService.queryQueue 方法。AdminMonitorService 解析队列类型（快充/慢充），通过 ChargingRequestMapper.findByQueueType() 查询对应等待队列中的所有请求，并调用 QueueService 计算队列位置和等待时间，最后返回 car_Id、car_Capacity、Request_Amount 和 waitTime 组成的队列状态列表。

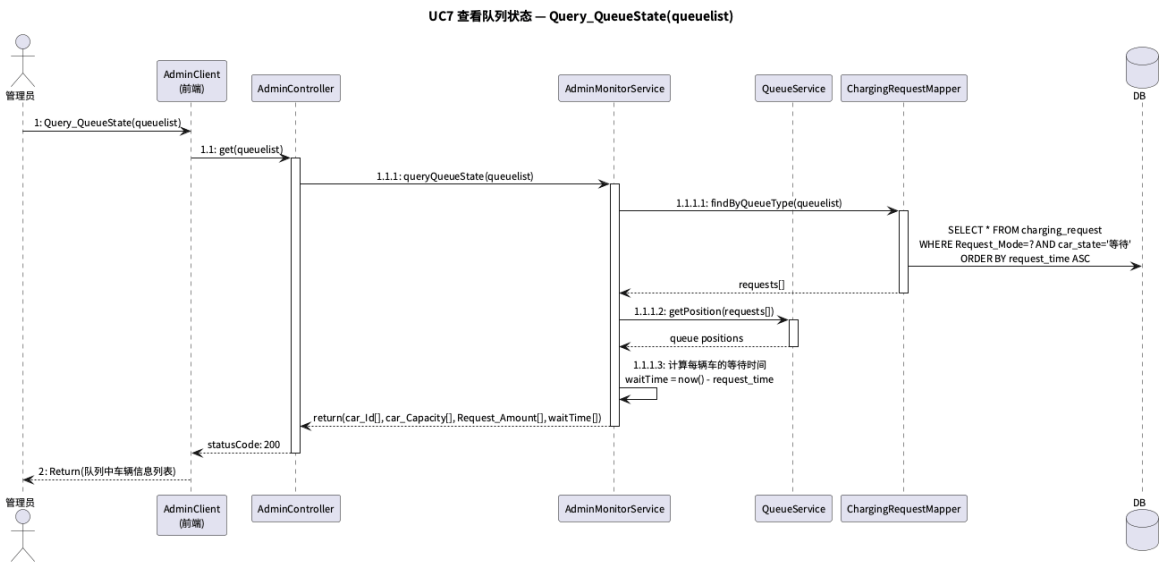


图 25: Query_QueueState 交互图

3.8 充电桩故障调度策略

当充电桩在运行过程中发生故障时，系统需要对受影响的车辆执行再调度。系统首先根据故障桩类型确定受影响车辆的充电模式：快充桩故障时，车辆只在快充可用桩集合内重新调度；慢充桩故障时，车辆只在慢充可用桩集合内重新调度。若同类型可用桩不足，车辆进入对应模式等待队列，不跨类型分配。以下给出三种调度策略的交互设计。

3.8.1 优先级调度策略

当充电桩发生故障时，系统先读取故障桩类型，并在与该类型一致的候选充电桩集合内对受影响车辆进行再调度。若多个故障或批量再调度同时涉及快充与慢充车辆，则快充请求优先于慢充请求；同类型请求内部按原始请求时间（request_time）升序排列（先到者优先）。对每辆受影响车辆，仅在同类型可用充电桩中选择等待时间最短的桩进行再分配，不跨类型占用充电桩。

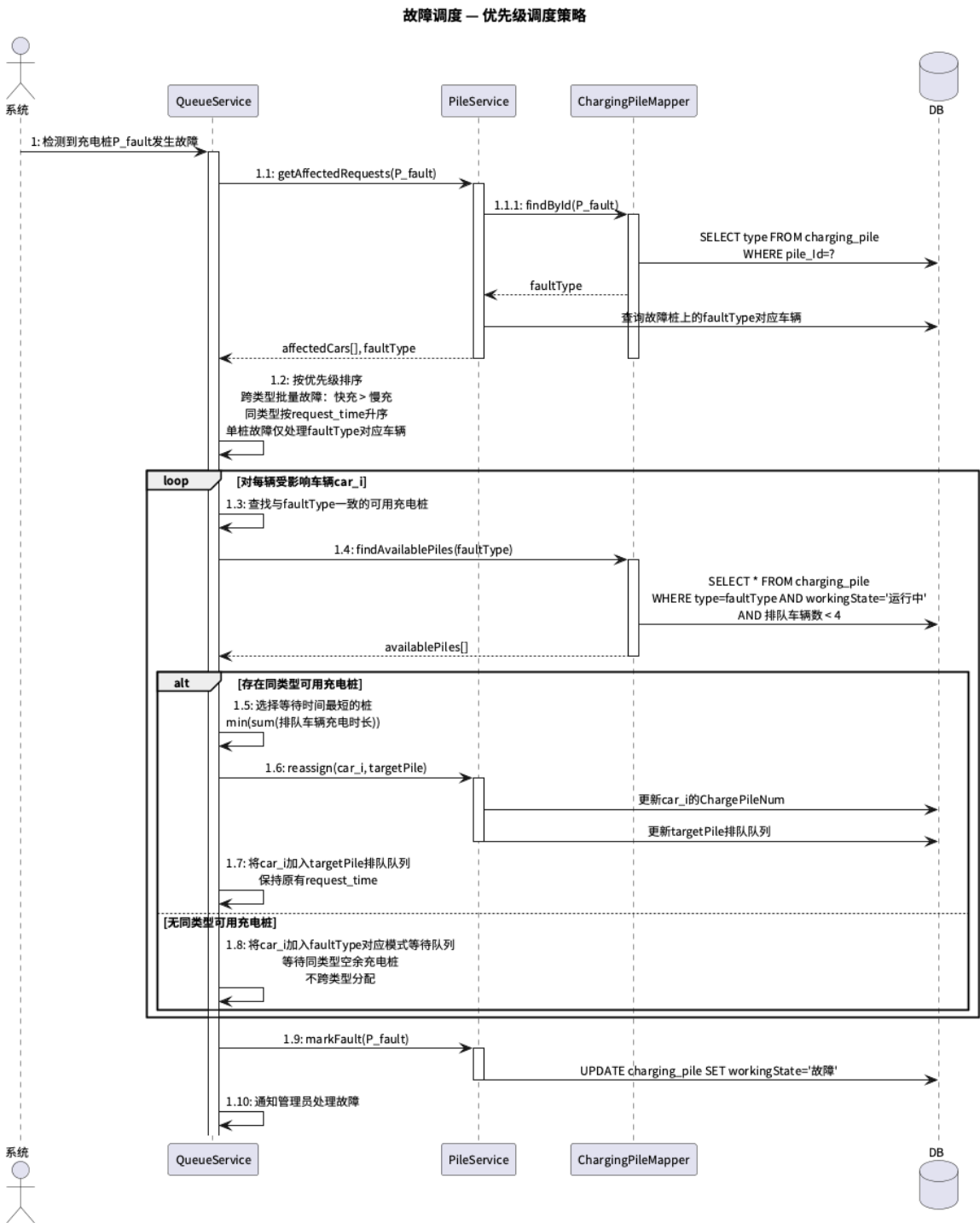


图 26: 故障调度——优先级调度策略交互图

3.8.2 时间顺序调度策略（FCFS）

当充电桩发生故障时，系统严格按照请求时间先后顺序（FCFS）对受影响车辆进行再调度。时间顺序调度只改变同类型车辆的处理顺序，不改变充电模式约束。故障桩为快充桩时，仅在快充可用桩中按 FCFS 重新分配；故障桩为慢充桩时，仅在慢充可用桩中按 FCFS 重新分配。若同类型可用桩不足，车辆进入对应模式等待队列。

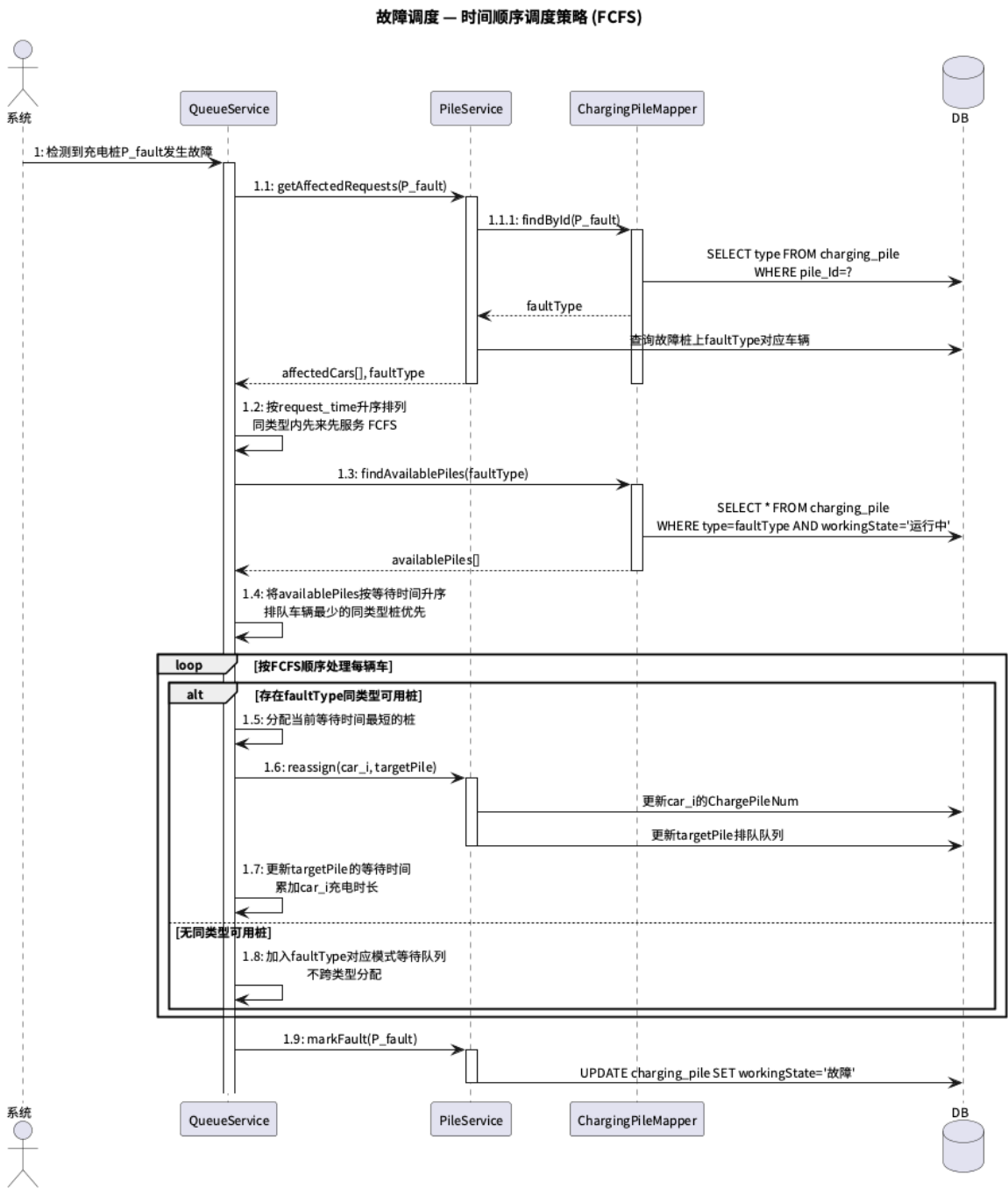


图 27：故障调度——时间顺序调度策略交互图

3.8.3 充电中故障恢复策略

当充电桩在充电过程中发生故障（有车辆正在充电），系统需要特殊处理该车辆的再调度。若故障可快速修复（如 5 分钟内），保持该车辆在原桩排队队列首位等待修复后继续充电（从断点电量继续）；若故障需长时间修复，则将该车辆以最高优先级分配到其他同类型可用充电桩。若没有同类型可用桩，则车辆保持最高优先级进入对应模式等待队列，不能分配到另一类型充电桩。

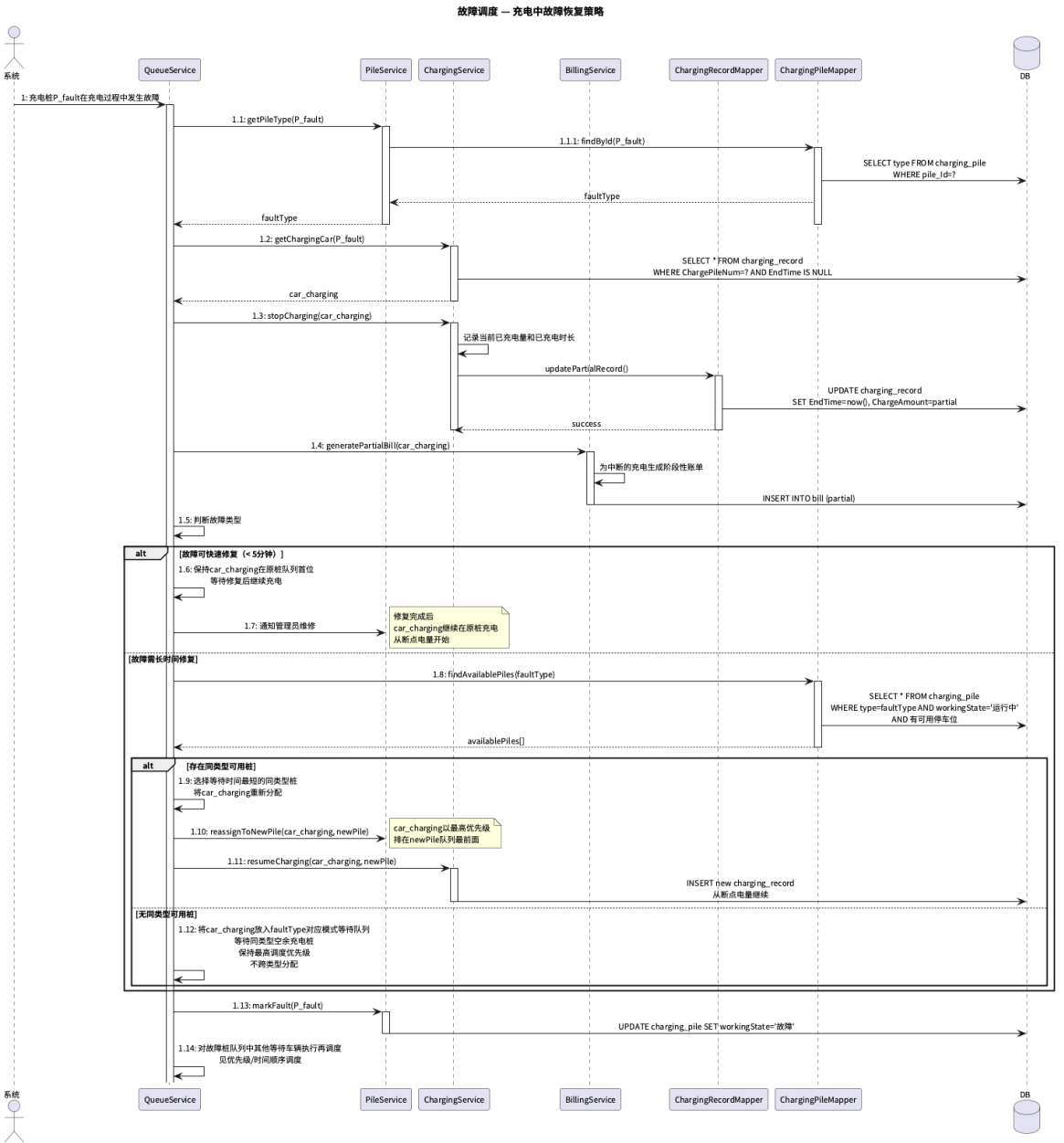


图 28: 故障调度——充电中故障恢复策略交互图

3.8.4 可选加分：单次调度总充电时长最短策略

当有空余充电桩触发调度时，系统遍历等待队列中与该充电桩同模式的所有车辆，选取“自身充电时长最短”的车辆进行分配。快充桩空闲时只遍历快充等待队列，慢充桩空闲时只遍历慢充等待队列。自身充电时长 = Request_Amount / 充电功率，目标是最小化单辆车的总完成时间。

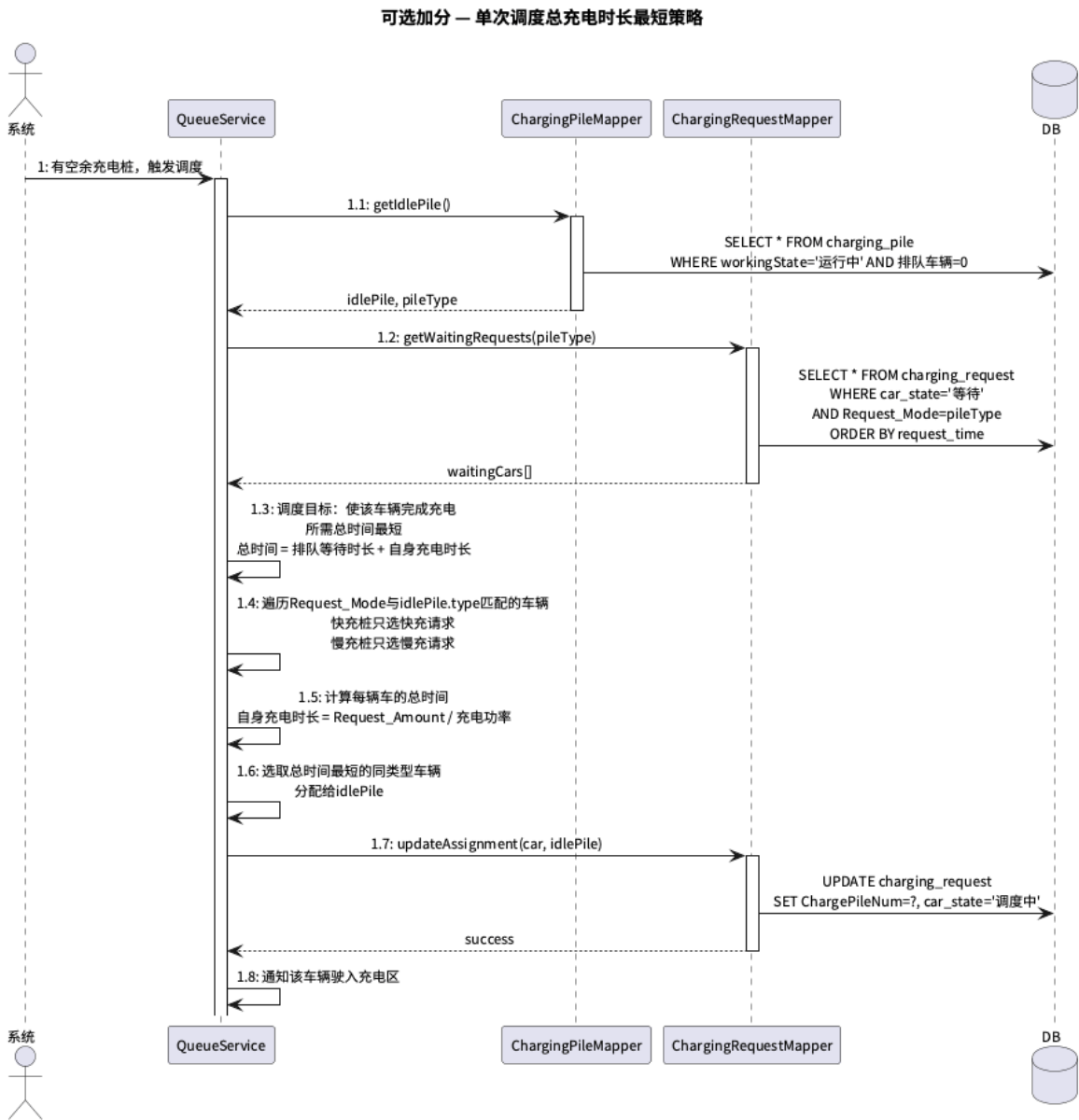


图 29: 可选加分——单次调度总充电时长最短策略交互图

3.8.5 可选加分：批量调度总充电时长最短策略

当多个充电桩同时空闲（或定时触发批量调度），系统采用贪心算法求解最优分配方案。构造代价矩阵时，仅对车辆充电模式与充电桩类型匹配的组合计算代价：若 Request_Mode(i) 与 type(pile_j) 一致，则 Matrix[i][j] = 车辆 i 在充电桩 j 上完成充

电的总时间；若二者不匹配，则该组合记为不可分配，不参与选择。系统迭代选取代价最小的合法（车辆，充电桩）对进行分配，使所有车辆的总等待时间最短。

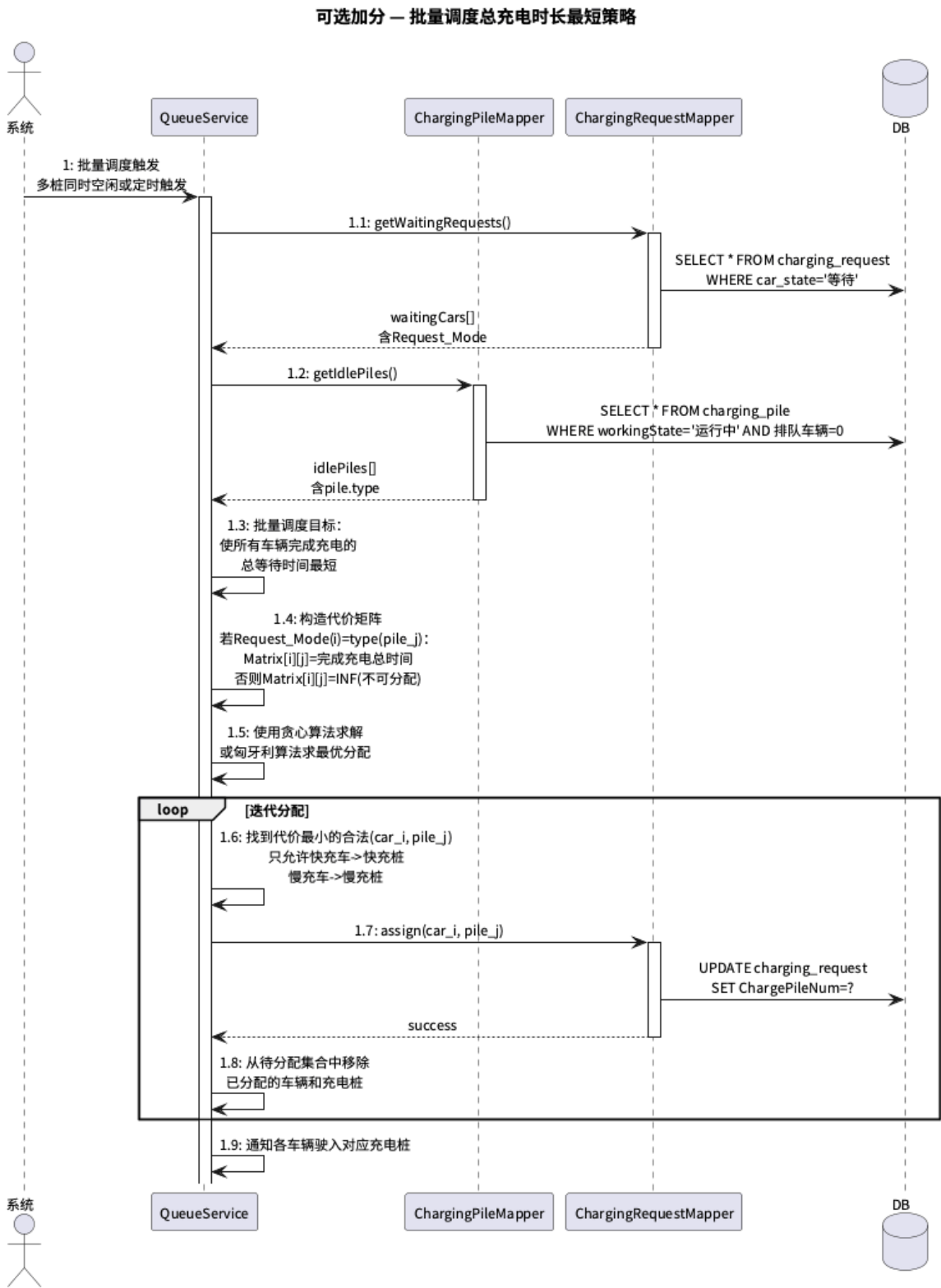


图 30: 可选加分——批量调度总充电时长最短策略交互图

4 系统静态结构设计

本章静态结构设计将用例实现涉及的软件对象划分为用户界面层、控制器层、业务/应用层和持久化层。其中，用户界面层表示用户端或管理员端界面；控制器层负责接收系统事件并协调其他对象；业务/应用层包含实现用例系统级功能的 Service 对象、状态机/策略对象，以及表示业务数据和业务状态的对象；持久化层包含负责对业务对象进行增删改查和数据库同步的 Mapper 对象及 SQLite 数据库。系统层主要由 JVM、SpringBoot 运行时、SQLite JDBC 驱动和操作系统文件系统提供支撑，在用例级类图中不单独展开。

4.1 UC1 注册

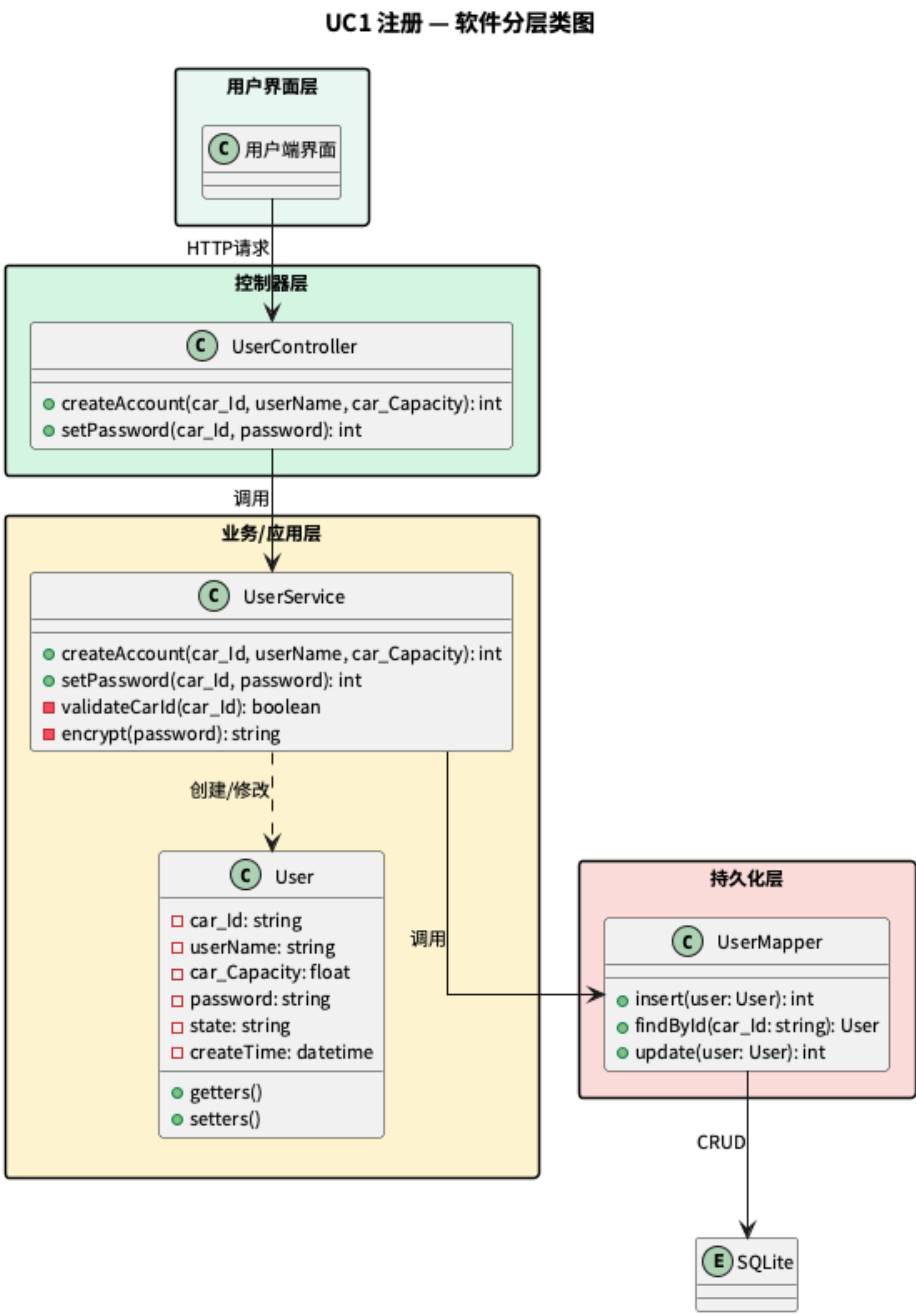


图 31: UC1 注册软件分层类图

表 8: UC1 注册类属性与方法说明

类	属性/方法	说明
UserController	createAccount() setPassword()	接收前端注册请求，创建账号 接收前端设置密码请求
UserService	createAccount() setPassword() encrypt()	创建 User 对象，校验 car_Id 唯一性 加密并设置密码，激活用户 密码加密方法
User	car_Id userName car_Capacity password state	车辆 ID（主键） 用户名 车辆电池容量 加密后的密码 用户状态（未激活/已激活）
UserMapper	insert() findById() update()	插入新用户记录 按 car_Id 查询用户 更新用户记录

4.2 UC2 充电申请

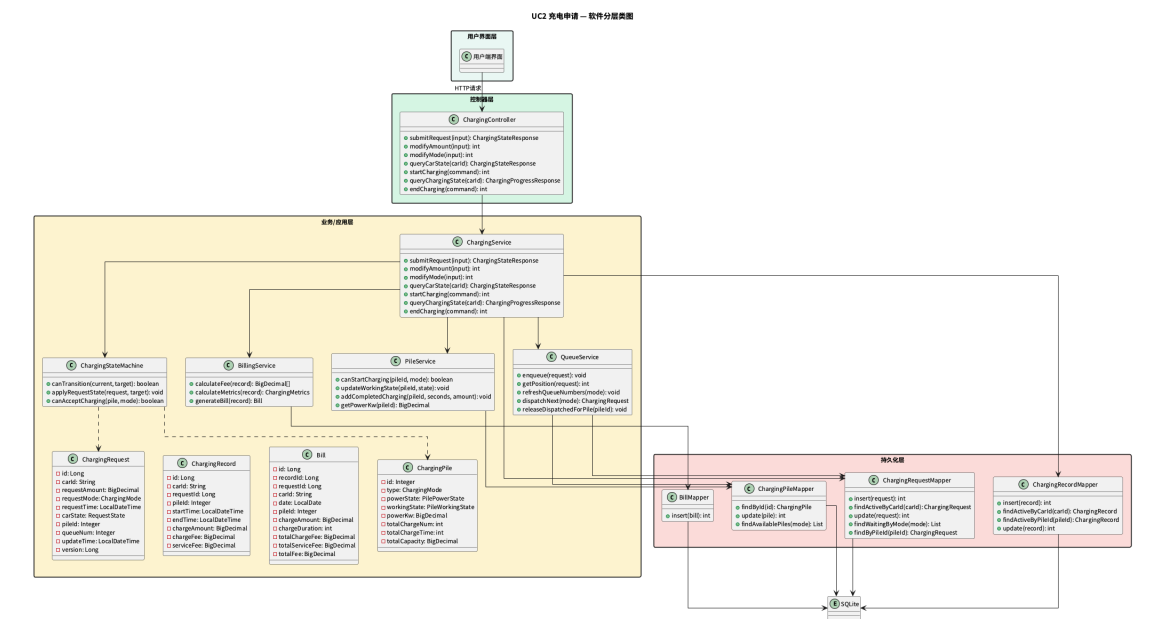


图 32: UC2 充电申请软件分层类图

表 9: UC2 充电申请类属性与方法说明

类	属性/方法	说明
Charging Controller	submitRequest() modifyAmount() modifyMode() queryCarState()	提交充电申请 修改充电电量 修改充电模式 查询车辆排队状态
ChargingSession Service	startCharging() queryCharging State() endCharging()	开始充电并创建 ChargingRecord 查询充电实时状态 结束充电并触发结算
QueueService	enqueue() getPosition() dispatchNext() releaseDispatched ForPile()	将请求加入等待队列 获取排队位置 调度下一辆车 释放已分配但未开始的请求
BillingService	calculateFee() generateBill()	实时计算充电费和服务费 生成充电账单
ChargingState Machine	applyRequestState()	维护请求状态迁移规则
ChargingRequest	carId requestAmount requestMode requestTime carState pileId / queueNum	车辆 ID 期望充电电量 充电模式 请求时间 车辆状态 分配的充电桩编号和队列序号
ChargingRecord	carId pileId startTime endTime chargeAmount	车辆 ID 充电桩编号 充电开始时间 充电结束时间 实际充电电量
Bill	id recordId / requestId totalChargeFee totalServiceFee totalFee	账单编号 关联的充电记录和请求 总充电费 总服务费 总费用

4.3 UC3 查看账单

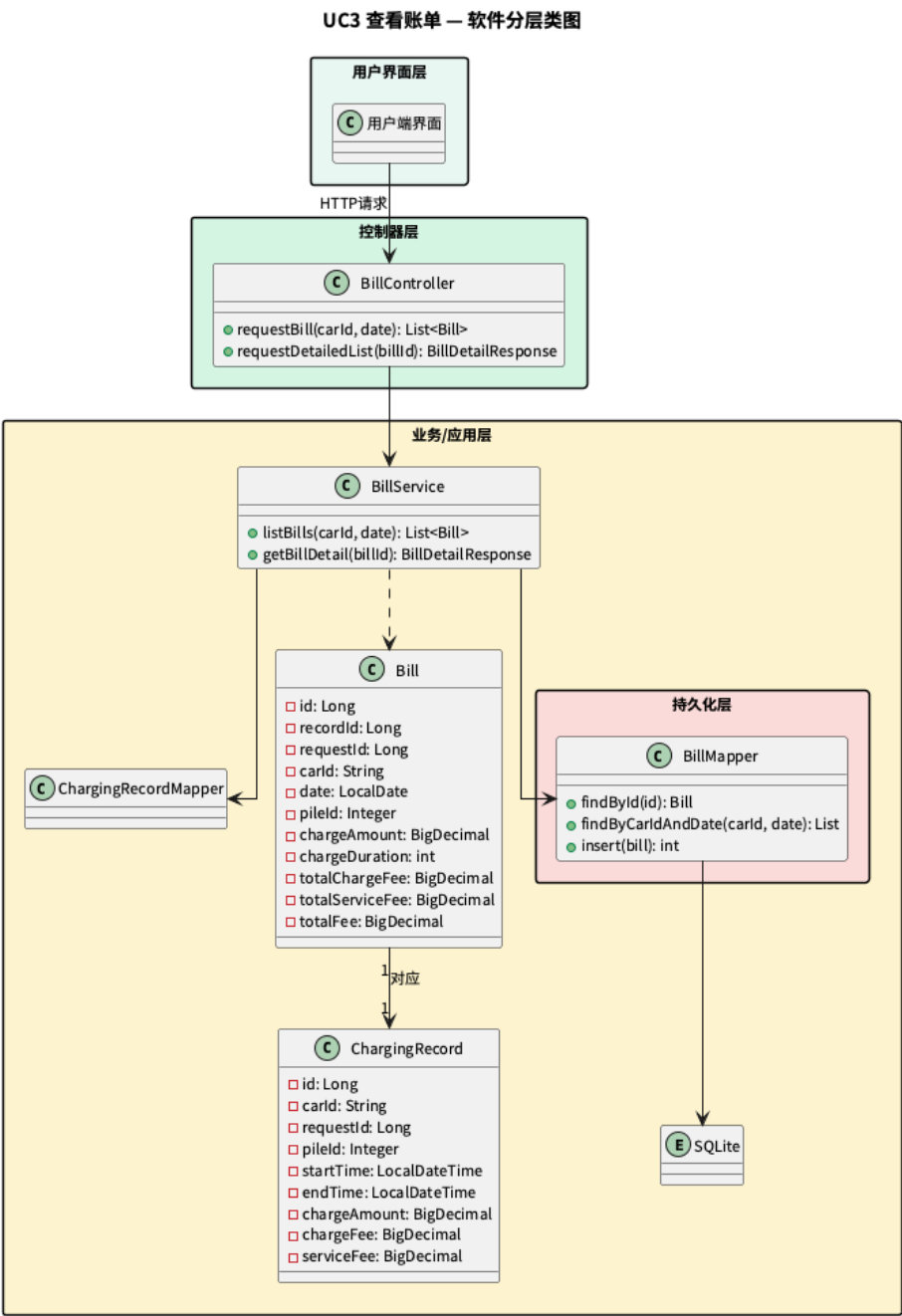


图 33: UC3 查看账单软件分层类图

表 10: UC3 查看账单类属性与方法说明

类	属性/方法	说明
BillController	requestBill()	接收查看账单列表请求
BillService	listBills()	按车辆和日期查询账单列表
Bill	id carId date totalFee	账单 ID 车辆 ID 账单日期 总费用
BillMapper	findByCarId AndDate()	查询账单列表

4.4 UC4 查看详单

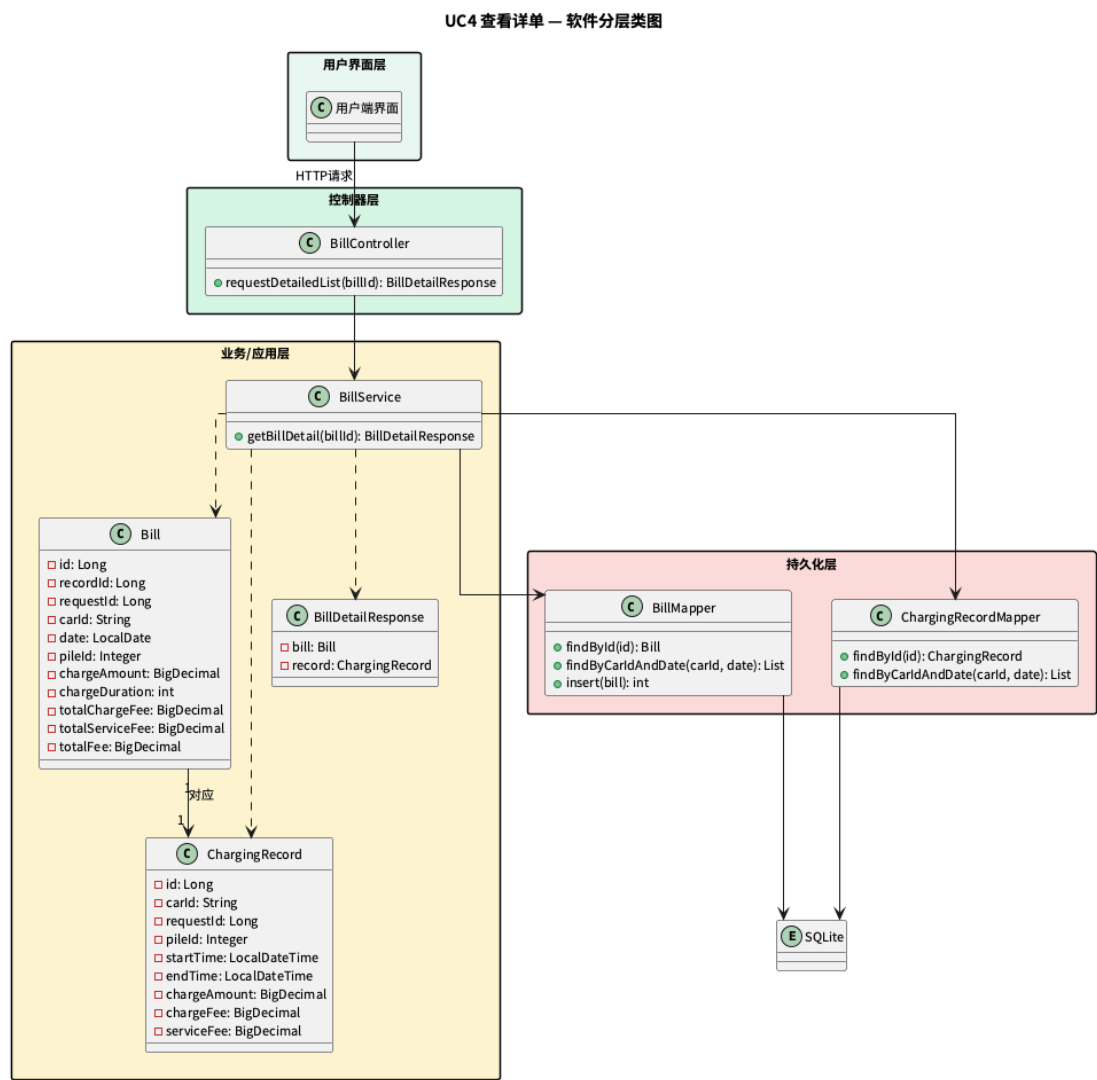


图 34: UC4 查看详单软件分层类图

表 11: UC4 查看详单类属性与方法说明

类	属性/方法	说明
BillController	requestDetailedList()	接收查看详单请求
BillService	getBillDetail()	查询账单及对应充电记录明细
BillDetail Response	bill / record	返回账单摘要和充电记录明细
ChargingRecord Mapper	findById()	按记录编号查询充电记录

4.5 UC5 运行充电桩

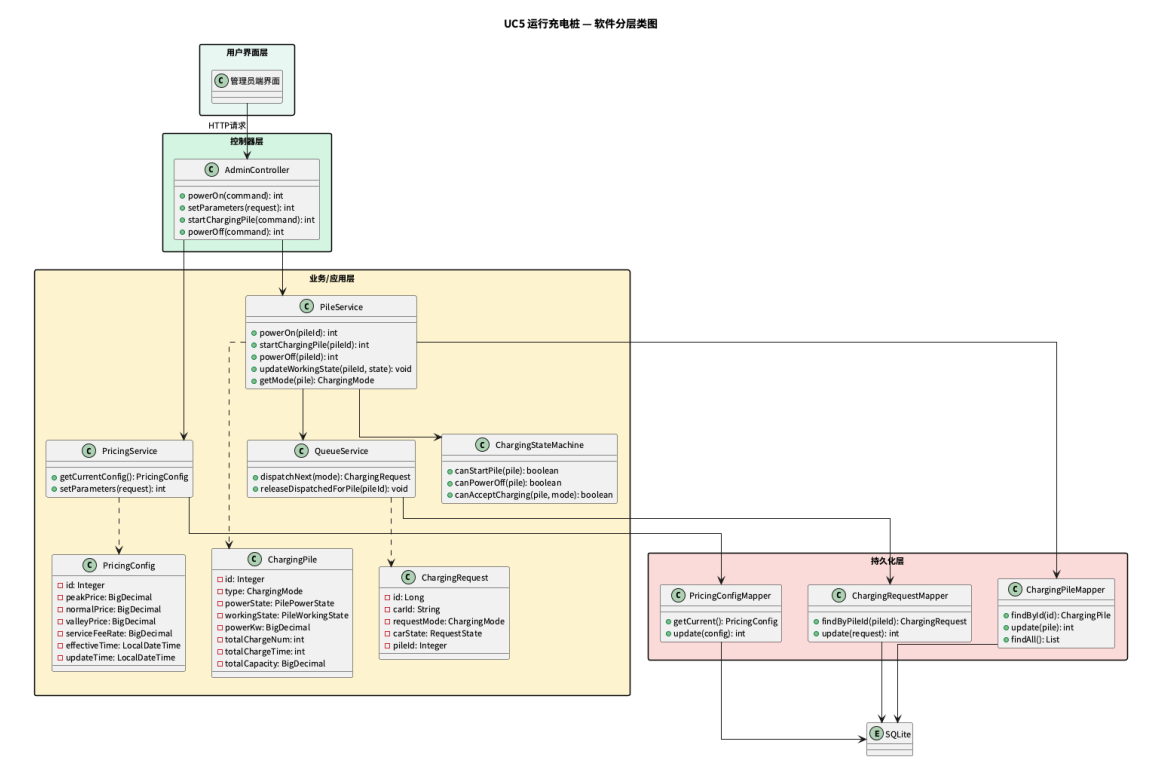


图 35：UC5 运行充电桩软件分层类图

表 12：UC5 运行充电桩类属性与方法说明

类	属性/方法	说明
AdminController	powerOn() setParameters() startCharging Pile() powerOff()	启动充电桩 设置计费参数 运行充电桩 关闭充电桩
PileService	powerOn() startCharging Pile() powerOff()	设置开机状态并触发调度 设置运行中 设置关机并释放分配请求
PricingService	setParameters()	更新计费参数
ChargingPile	type / powerState / workingState	保存充电桩类型、电源状态和 运行状态

4.6 UC6 查看充电桩状态

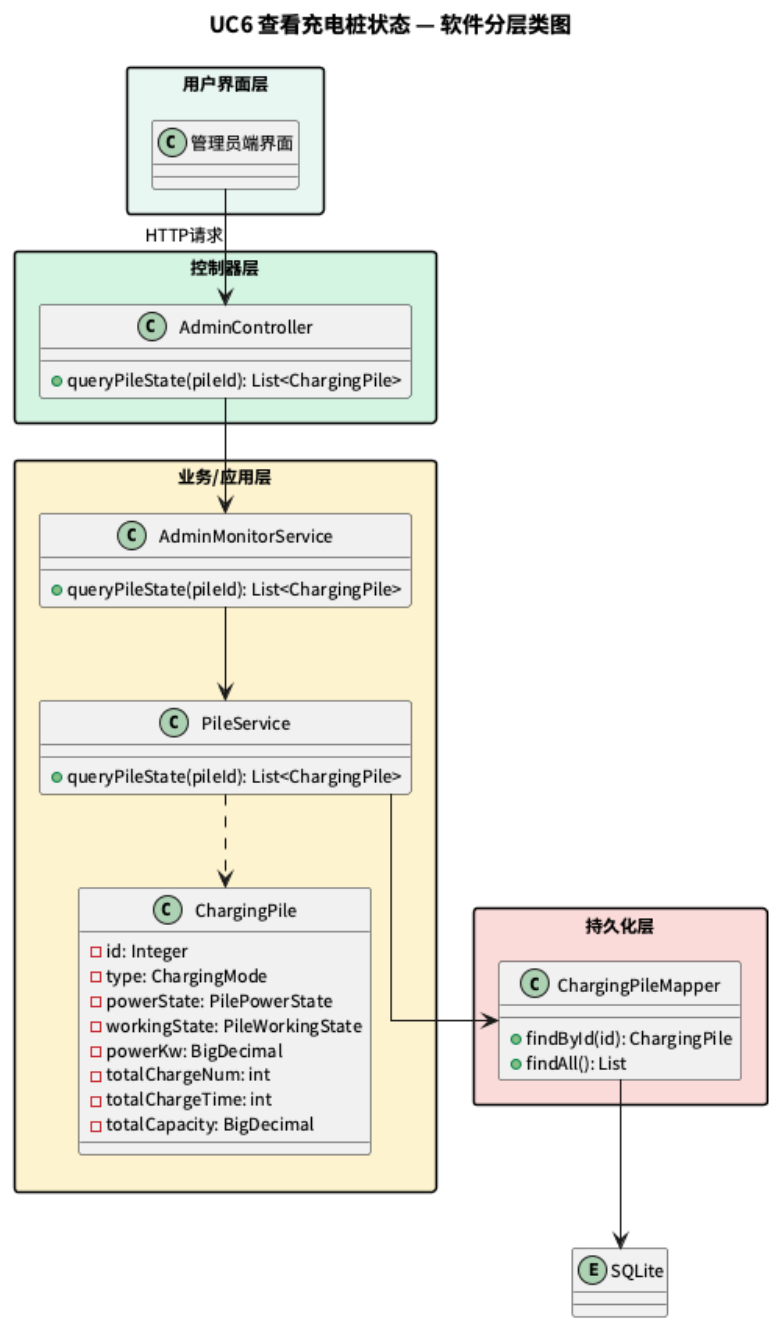


图 36: UC6 查看充电桩状态软件分层类图

表 13: UC6 查看充电桩状态类属性与方法说明

类	属性/方法	说明
AdminController	queryPileState()	接收充电桩状态查询请求
AdminMonitorService	queryPileState()	聚合查询充电桩运行状态
PileService	queryPileState()	提供充电桩状态查询，查询结果包含 type 以区分快充桩和慢充桩

4.7 UC7 查看队列状态

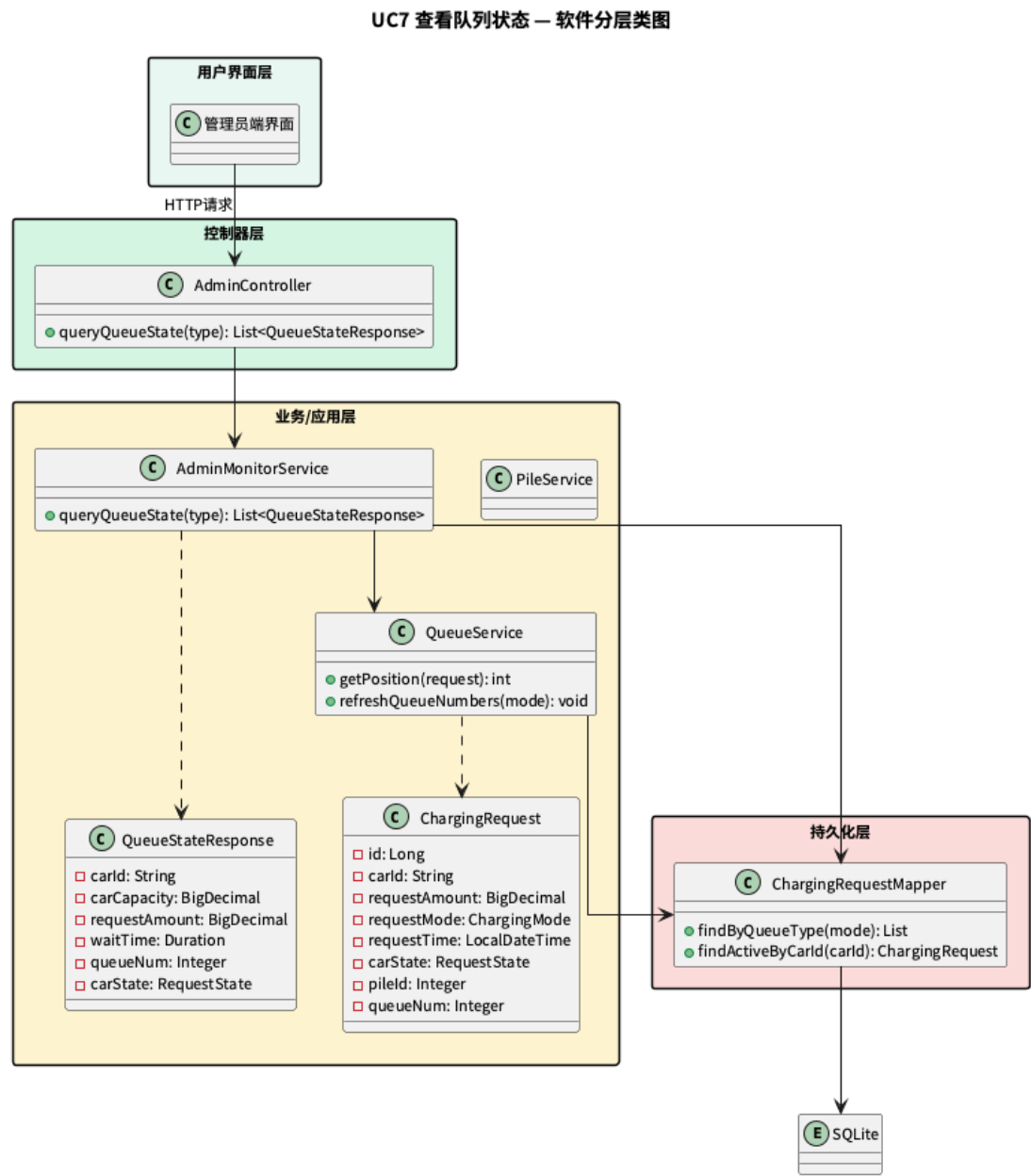


图 37: UC7 查看队列状态软件分层类图

表 14: UC7 查看队列状态类属性与方法说明

类	属性/方法	说明
AdminController	queryQueueState()	接收队列状态查询请求
AdminMonitorService	queryQueueState()	聚合查询快充/慢充等待队列
QueueService	getPosition()	计算队列位置和排队序号
ChargingRequestMapper	findByQueueType()	查询指定模式等待队列

4.8 系统级静态结构

为补充前述用例级静态结构，本节给出系统级分层静态结构。该图从整体角度展示用户界面层、控制器层、业务/应用层、持久化层与系统层之间的对象依赖关系，用于说明各用例涉及的软件对象在完整系统中的组织方式。系统级类图不替代各用例级类图，而是对核心控制对象、业务对象、持久化对象和运行支撑对象进行统一归纳。

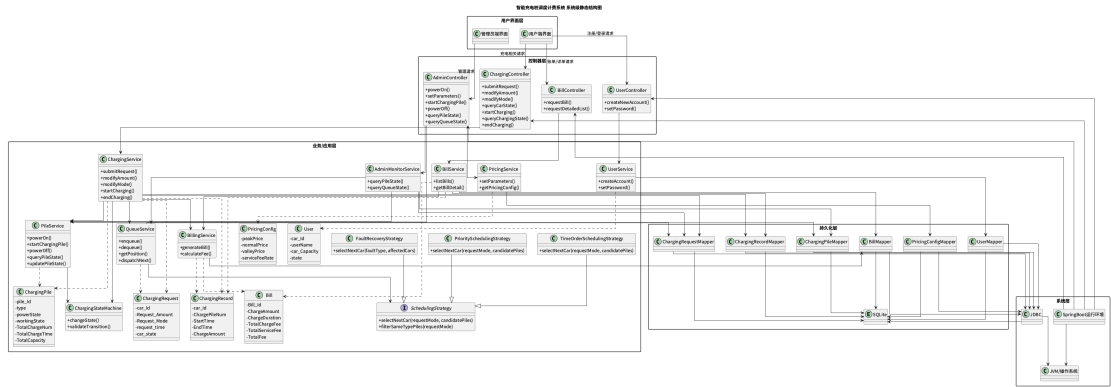


图 38: 系统级软件分层类图

表 15: 系统级类属性与方法说明

类	属性	方法与说明
用户端界面	currentUser、 currentRequest、 queueState、 chargingState、 billList	submitChargingRequest() 提交充电申请； queryCarState() 查看排队状态； queryChargingState() 查看充电状态； requestBill() 查询账单； requestDetailed request() 查询详单。
管理员端界面	pileStateList、 queueStateList、 pricingConfig	powerOn() 启动充电桩； setParameters() 设置计费参数； startChargingPile() 运行充电桩； powerOff() 关闭充电桩； queryPileState() 查看桩 queryQueue 状态； State() 查看队列状态。

类	属性	方法与说明
UserController	userService	createNewAccount() 接收注册请求; setPassword() 接收密码设置请求; 负责参数校验并调用 UserService。
ChargingController	chargingService	submitRequest() 提交充电申请; modifyAmount() 修改充电量; modifyMode() 修改充电模式; queryCarState() 查询车辆排队状态; startCharging() 开始充电; queryChargingState() 查看充电状态; endCharging() 结束充电。
BillController	billService	requestBill() 按车辆和日期查询账单; requestDetailedList() 根据 Bill_Id 查询详单。
AdminController	pileService、pricingService、adminMonitorService	powerOn() 启动充电桩; setParameters() 设置计费规则; startChargingPile() 运行充电桩; powerOff() 关闭充电桩; queryPileState() 和 queryQueueState() 提供监控查询入口。
UserService	userMapper	createAccount() 创建 User 对象并初始化车辆容量、用户名和状态; setPassword() 加密并保存密码, 修改用户状态。
ChargingService	chargingRequestMapper、chargingRecordMapper、queueService、billingService、pileService、stateMachine	submitRequest() 创建 ChargingRequest; modifyAmount() 修改充电量; modifyMode() 调整充电模式和队列; startCharging() 创建 ChargingRecord 并修改桩状态; endCharging() 结束充电并触发计费和调度。

类	属性	方法与说明
QueueService	chargingRequestMapper、 schedulingStrategy	enqueue() 按 Request_Mode 将请求加入快充或慢充等待队列；dequeue() 移出队列；getPosition() 计算排队位置；dispatchNext() 在同类型空闲充电桩范围内选择下一辆车。
BillingService	billMapper、 pricingService	generateBill() 根据充电记录创建 Bill 对象；calculateFee() 按峰平谷电价和服务费率计算总费用。
BillService	billMapper、 chargingRecordMapper、 billingService	listBills() 按车辆和日期返回账单列表；getBillDetail() 根据 Bill_Id 组装详单返回信息。
PileService	chargingPileMapper、 stateMachine	powerOn() 修改充电桩电源状态；startChargingPile() 设置运行状态；powerOff() 关闭充电桩；queryPileState() 查询运行统计；updatePileState() 同步状态变化。
PricingService	pricingConfigMapper	setParameters() 创建或修改 PricingConfig；getPricingConfig() 读取当前计费参数。
AdminMonitor Service	pileService、 queueService、 chargingPileMapper、 chargingRequestMapper	queryPileState() 聚合充电桩运行状态；queryQueueState() 聚合快充/慢充等待队列状态。
ChargingState Machine	stateTransitionRules	changeState() 执行车辆请求或充电桩状态迁移；validateTransition() 检查状态迁移是否合法。
SchedulingStrategy	无固定属性	selectNextCar() 定义同类型候选集合内的车辆选择规则，不改变快充/慢充匹配约束。

类	属性	方法与说明
PrioritySchedulingStrategy	priorityRules	selectNextCar() 在同类型候选集合内按优先级规则选择待充车辆。
TimeOrderSchedulingStrategy	requestTimeOrder	selectNextCar() 在同类型候选集合内按请求时间顺序选择待充车辆。
FaultRecoveryStrategy	faultPile、affectedCars	selectNextCar() 在充电桩故障后按故障桩类型重新选择和迁移受影响车辆，不跨类型分配。
User	car_Id、userName、car_Capacity、password、state	getCarId()、setPassword()、setState() 维护用户账号、车辆容量和账号状态。
ChargingRequest	car_Id、Request_Amount、Request_Mode、request_time、car_state、queue_Num	setRequestAmount()、setRequestMode()、setCarState() 维护充电请求和排队状态。
ChargingRecord	car_id、ChargePileNum、StartTime、EndTime、ChargeAmount	setStartTime()、setEndTime()、setChargeAmount() 记录一次充电过程。
Bill	Bill_Id、carId、date、ChargePileNum、ChargeAmount、ChargeDuration、TotalChargeFee、TotalServiceFee、TotalFee	calculateTotalFee()、getBillDetail() 保存账单汇总和详单查询所需信息。
ChargingPile	pile_Id、type、powerState、workingState、TotalChargeNum、TotalChargeTime、TotalCapacity	setPowerState()、setWorkingState()、updateStatistics() 维护充电桩类型、开关状态、运行状态和累计统计。

类	属性	方法与说明
PricingConfig	peakPrice、normalPrice、valleyPrice、serviceFeeRate	updatePrices()、getPriceByTime() 维护峰平谷电价和服务费率。
UserMapper	user 表映射	insert()、findById()、findByCarId()、update() 保存和查询 User 对象。
ChargingRequestMapper	charging_request 表映射	insert()、findByCarId()、findByQueueType()、update()、delete() 保存和查询充电请求。
ChargingRecordMapper	charging_record 表映射	insert()、findActiveByCarId()、findById()、update() 保存和查询充电记录。
BillMapper	bill 表映射	findById()、insert()、AndDate()、findById() 保存和查询账单。
ChargingPileMapper	charging_pile 表映射	findById()、findAll()、findIdlePile()、update() 保存和查询充电桩状态。
PricingConfigMapper	pricing_config 表映射	findCurrent()、insert()、update() 保存和读取计费参数。
SQLite	user、charging_request、charging_record、bill、charging_pile、pricing_config 数据表	executeQuery()、executeUpdate() 提供本地关系型数据持久化存储。
SpringBoot 运行环境	controllerRegistry、beanContainer、httpServer	registerController()、injectDependency()、handleRequest() 提供控制器注册、依赖注入和 HTTP 请求处理。
JDBC	connection、preparedStatement、resultSet	getConnection()、executeQuery()、executeUpdate() 为 Mapper 对象提供数据库访问接口。

类	属性	方法与说明
JVM/操作系统	process、thread、fileSystem、networkStack	startProcess()、scheduleThread()、readFile()、writeFile() 支撑 Java 程序运行、文件访问和网络通信。

5 工作量统计

表 16: 作业工作内容及工作量统计

模块	工作内容	宋旗舰	黄俊恺	张艺鸿	龙康维	谭怡萱
软件架构	架构设计与分层说明	✓				
界面设计	用户充电控制面板		✓			
界面设计	管理员管理界面			✓		
UC1 注册	createNewAccount set_pwd				✓	✓
UC2 充电申请	E_chargingRequest	✓				
	Modify_Amount		✓			
	Modify_Mode			✓		
	Query_Car_State				✓	
	Start_Charging					✓
	Query_Charging_State	✓				
	End_Charging		✓			
UC3 查看账单	Request_Bill			✓		
UC4 查看详单	Request_DetailedList				✓	
UC5 运行充电桩	powerOn					✓
	setParameters	✓				
	Start_ChargingPile		✓			
	powerOff			✓		
UC6 查看充电桩状态	Query_PileState				✓	
UC7 查看队列状态	Query_QueueState					✓
故障调度 (组长)	优先级/时间顺序/故障恢复	✓				
加分项	单次/批量调度策略	✓				
静态结构设计	UC1 类图		✓			
静态结构设计	UC2 类图	✓				
静态结构设计	UC3 类图			✓		
静态结构设计	UC4 类图				✓	
静态结构设计	UC5 类图					✓
静态结构设计	UC6 类图	✓				
静态结构设计	UC7 类图		✓			